EMF Envelope FFT Analyzer

User Manual

Envelope (modulation) spectra of broadband EMF probe logs
(FieldMan / Narda-TSX, NBM-550, EHP-50 TS) and SignalShark spectrograms

Program version 0.0.4 · vszh5_writer 0.0.13 · 2026-10-15

William W. Wallace
NRAO/GBO
wwallace@nrao.edu

Copyright © 2026 William W. Wallace (Adaptive Electrodynamics LLC dba AdaptEd).
This manual and the programs it describes are free software under the
GNU General Public License v3.0 or later (GPL-3.0-or-later).

Internal draft: tested on synthetic files only (Section 12).

Contents

1 Introduction **5**
1.1 What the analysis can and cannot tell you . . . 5
1.2 Status of this version . . . 5
1.3 Conventions . . . 5

2 Installation **6**
2.1 Mamba environment . . . 6
2.2 Package files . . . 6
2.3 Veusz and LuaLaTeX (optional) . . . 6
2.4 Spyder . . . 7

3 Quick start **7**
3.1 Typical workflow . . . 7

4 The graphical user interface **8**
4.1 Starting the GUI . . . 8
4.2 Window layout . . . 8
4.3 Units, display and themes . . . 10
4.4 Menus and keyboard shortcuts . . . 10
4.5 About dialog . . . 11

5 Instruments and input files **14**
5.1 Probe catalogue . . . 14
5.2 Readers . . . 14
5.3 How text exports are read . . . 14
5.4 EHP-50 TS spectra . . . 15
5.5 SignalShark files . . . 15
5.6 Overriding the detection . . . 16

6 Methods: theory and implementation **16**
6.1 Square-law detection adds powers . . . 16
6.2 Why the transform is taken of power . . . 17
6.3 Units, conversions and dB conventions . . . 17
6.4 Sampling, probe averaging and aliasing . . . 20
6.5 Putting the log on a uniform grid . . . 21
6.6 Windows . . . 21
6.7 FFT amplitude spectrum . . . 22
6.8 Welch power spectral density . . . 23
6.9 Chirp-Z zoom . . . 23
6.10 Lomb–Scargle periodogram . . . 24
6.11 Envelope spectrogram (short-time FFT) . . . 25
6.12 Probe correction . . . 26
6.13 Peaks and their exact frequency . . . 26
6.14 Harmonic families and rotation rate . . . 26
6.15 Spectrograms: envelope spectrum per frequency bin and channels . . . 27

7 Command-line reference **27**

8 Output files **28**
8.1 The Veusz document . . . 29
8.2 Several units in the Veusz output . . . 30
8.3 Rendering to PDF and PNG . . . 31
8.4 Summary file . . . 31
8.5 Examples . . . 31

9 Live mode **38**

10 Memory, parallel processing and GPU **38**

11 Working in Spyder **39**

12 Verification **40**
12.1 Verification of the transforms and unit conversions 41
12.2 Further checks 43

13 Limitations and known issues **43**

14 Troubleshooting **44**

A Python modules used **45**
A.1 NumPy 45
A.2 SciPy 46
A.3 pandas and openpyxl 46
A.4 h5py 47
A.5 vszh5_writer (the author's module, 0.0.13) 47
A.6 Qt.py, QtPy, PySide6 and pyqtgraph 48
A.7 CuPy (optional) 49
A.8 pypdf and Veusz (rendering only) 50
A.9 Python standard library 50

B Revision history **51**

References **52**

List of Tables

1 Files of the package. . . . 6
2 Settings panel groups and the matching command-line options. . . . 9
3 Result tabs of the Analysis tab. . . . 9
4 Menu commands of the main window (read from the running window). . . . 10
5 Probe catalogue (datasheet values) and the derived envelope limits. . . . 14
6 Readers and how the input is recognised. . . . 14
7 Keywords that select columns (case ignored). . . . 15
8 The methods at a glance. . . . 16
9 Power-density input units: W/m^2 per unit (reading × factor = W/m^2). . . . 17
10 Analysed quantities, their origin and their unit family (`QUANTITY_FAMILY`); the last column is the default display unit. . . . 18
11 Display units (`DISPLAY_UNITS`, also `--list-units`): the value shown for a value held in the family's base unit, and the level rule. . . . 19
12 Time-plot modes (`--time-display`) and RF-level modes (`--rf-level-display`). . . . 20
13 Window properties computed from the SciPy windows (N = 4096, periodic): coherent gain, equivalent noise bandwidth, scalloping loss at half a bin, highest side lobe, overlap correlation at 50 % and Welch variance factor $1 + 2c^2$. . . . 22
14 Command-line options (from the program's parser). . . . 27
15 Output files (folder `<stem>_fft`). . . . 29
16 Pages written for every time series (prefix `ch<k>_` for spectrogram channels). . . . 29
17 Overview pages of a spectrogram (SignalShark, EHP-50 TS), before the channel pages. 30
18 Pages of an extra unit section (layout sections), by the family of the extra unit. . . . 31
19 Spyder cells of emf_envelope_fft_v0_0_4.py. . . . 39
20 Self-test result (program 0.0.4, run during this build). . . . 40
21 Verification of transforms and units (program 0.0.4, run during this build). . . . 41
22 Troubleshooting. . . . 44
23 Python modules used by the analyzer. . . . 45
24 NumPy calls. . . . 45
25 SciPy calls. . . . 46
26 pandas calls. . . . 46
27 h5py calls. . . . 47
28 vszh5_writer calls. . . . 47
29 Qt.py, PySide6 and pyqtgraph calls. . . . 48
30 pyqtgraph calls. . . . 49
31 CuPy calls (only with –gpu). . . . 49
32 pypdf and Veusz (through vszh5_writer). . . . 50
33 Python standard library modules used. . . . 50
34 Revision history of the program, the writer module and this manual. . . . 51

List of Figures

1 Main window, light theme: settings dock (left), toolbar, result selector and tabs (amplitude spectrum of a FieldMan/EFD-4091 log with the probe-corrected trace, X/Y/Z axes and peak markers), log dock and status bar. *(Synthetic data.)* . . . 9
2 Units and display group (left, scrolled into view): one unit per quantity family, time-plot and RF-level modes; the amplitude spectrum in dBm/m^2 (a level unit, so a linear axis) and the toolbar S and E unit selectors. *(Synthetic data.)* . . . 12
3 Dark theme: modulation map of a SignalShark spectrogram channel. *(Synthetic data.)* 12
4 Help → About: author, version and the GPL-3.0-or-later licence. *(Synthetic data.)* . . . 13
5 Veusz summary page: input, units, timing, probe model, peaks, harmonic families and notes, in the chosen display unit (here mW/cm^2). *(Synthetic data.)* . . . 32
6 Veusz overview page: time series, amplitude spectrum, Welch PSD and chirp-Z zoom of one log. *(Synthetic data.)* . . . 32

7 Synthetic FieldMan/Narda-TSX log (EFD-4091): field against the instrument clock on a date axis. The 6 s gap at 300 s is deliberate. *(Synthetic data.)* . . . 33
8 Envelope amplitude spectrum of the same log: the 12 rpm radar family (0.2 Hz and harmonics), the 0.7 Hz source, the probe-corrected trace (dashed) and the X/Y/Z axes (dotted). *(Synthetic data.)* . . . 33
9 Chirp-Z zoom around the strongest line: the dense CZT samples (line) between the FFT bins (markers). *(Synthetic data.)* . . . 34
10 Probe integration model $|H(\nu)| = |\mathrm{sinc}(\nu T)|$ for $T = 265$ ms up to the Nyquist frequency. *(Synthetic data.)* . . . 34
11 Synthetic SignalShark spectrogram: two carriers whose levels vary at 3 Hz and 0.5 Hz. *(Synthetic data.)* . . . 35
12 Envelope spectrum of every RF bin of that spectrogram: each carrier shows its own modulation frequency. *(Synthetic data.)* . . . 35
13 Logged field of the same log shown as E in dBµV/m (display units `mW/cm2,dBuV/m`). *(Synthetic data.)* . . . 36
14 Page of an extra unit section (`--veusz-units dBm/m2`): the amplitude spectrum in dBm/m^2, $10\log_{10}$ of the amplitude in mW/m^2, on a linear axis. *(Synthetic data.)* . . . 36
15 Page of the `mA/m` section: $H = \sqrt{S/Z_0}$ of the equivalent plane wave against time. *(Synthetic data.)* . . . 37
16 Live tab (dark theme) following a growing file with 2 Hz and 7 Hz envelope tones. *(Synthetic data.)* . . . 38

1 Introduction

A broadband diode probe such as the Narda EFD-4091, EFD-0691, EF 0691 or EFD-9091 adds the power of every source in its frequency range and reports one number per sample [1, 2, 3]. Its log therefore holds no information about the radio frequency of a source. It does hold how the total field changes in time: a radar antenna rotating, a beam being scanned, a transmitter keyed on and off, a duty cycle. The *EMF Envelope FFT Analyzer* takes the Fourier transform of that envelope, the *modulation spectrum*, so that sources which share an RF band but change at different rates can be told apart and their rates measured (a rotating antenna appears as a line at its rotation frequency, with harmonics from the beam shape).

The program reads the files written by the software of the instruments in use: FieldMan and Narda-TSX exports for the EFD probes [5, 9], NBM-550 or NBM-TS logs for the EF 0691 probe [6], EHP-50 TS spectra of the EHP-50F low-frequency analyzer [7], and SignalShark spectrum or spectrogram files [8]. A SignalShark spectrogram, unlike a probe log, does resolve RF frequency; for it the envelope transform is taken for every frequency bin.

1.1 What the analysis can and cannot tell you

- **Can**: the rates at which the total field changes (rotation in rpm, scan or keying periods), the relative strength of each modulation, and which modulations come from the same source (harmonic families).
- **Cannot**: the RF frequency of a source measured with a diode probe; content faster than half the probe's sample rate (it folds back, Section 6.4); an exposure assessment, for which IEC 62232 [18] and calibrated probes (IEEE 1309 [17]) remain the reference.
- **Only partly**: content near the Nyquist frequency is attenuated by the probe's own averaging; the program models and can partly undo that (Section 6.12).

The program has a graphical user interface (Section 4), which opens when the script is run without arguments, and a command line for batch work. Every plot, table and Veusz page can be shown in the unit wanted for each kind of quantity (power density, E, H, B, port power or voltage, linear or in dB), and the Veusz output can carry several units at once (Section 6.3, Section 8.2); this works the same in post-processing and in live mode. Every transform and unit conversion is verified against analytic answers and independent implementations (Section 12).

1.2 Status of this version

Version 0.0.4 is an internal draft. No real export of FieldMan, Narda-TSX, NBM-TS, EHP-50 TS or SignalShark has been available yet, so the readers recognise file layouts by heuristics (Section 5) and were tested on synthetic files written in the expected styles. Use `--inspect` on every new kind of file before trusting an analysis, and see Section 13 for the known limitations.

1.3 Conventions

- f is an RF (or spectrum) frequency, ν an envelope (modulation) frequency; f_s is the sample rate of the log, N the number of samples, and T the probe's integration time.
- $\operatorname{sinc} x = \sin(\pi x)/(\pi x)$, the normalised form used by NumPy.
- Versions follow Semantic Versioning (`X.Y.Z`); the manual carries the version of the program it documents.
- Commands are shown for a Windows or Linux shell inside the mamba environment; `python` is the environment's interpreter.

2 Installation

2.1 Mamba environment

The environment file creates an environment named `emf-fft` from conda-forge with Python 3.12 and every dependency [36]:

```
mamba env create -f emf_envelope_fft.yml
mamba activate emf-fft
python emf_envelope_fft_v0_0_4.py --selftest
```

The self-test (Section 12) must end with `SELF-TEST PASSED`; it includes the verification of every transform and unit conversion (also on its own: `--verify`). Then start the GUI with `python emf_envelope_fft_v0_0_4.py`. For a GPU (NVIDIA CUDA only), also install CuPy (`mamba install -c conda-forge cupy`) and add `--gpu`.

2.2 Package files

Keep the files together as delivered; `vszh5_writer.py` must sit in the same folder as the script, which puts its own folder first on the module search path so that this copy is used rather than an older one (e.g. from the QuickSyn package). Table 1 lists the package.

Table 1: Files of the package.

File	Type	Size	Description
`emf_envelope_fft_v0_0_4.py`	Python 3.12 script	6920 lines	The analyzer: readers, estimators, Veusz/CSV/JSON output, batch, live mode, self-test. Spyder cells (Table 19).
`vszh5_writer.py`	Python module	1402 lines	The author's Veusz HDF5 writer, 0.0.13: typed datasets, 2-D images, plot, grid and text pages, reading a document back, rendering to PDF (with bookmarks) and PNG. Must stay next to the script.
`emf_envelope_fft.yml`	mamba environment	–	Creates the environment emf-fft from conda-forge (Python 3.12 and every dependency; CuPy commented out).
`LICENSE`	text	–	GNU General Public License v3.0.
`docs/EMF_Envelope_FFT_Manual_v0.0.4.pdf`	PDF	–	This manual, with heading-based bookmarks.
`docs/EMF_Envelope_FFT_Manual_v0.0.4.tex`	LuaLaTeX source	–	Source of this manual; written by the build tool, compiles on its own with lualatex.
`docs/figures/*.png`	PNG	–	Figures of this manual: Veusz pages rendered from the self-test's synthetic files (also in extra unit sections), and screenshots of the GUI (light, units, dark, Live tab, About).
`tools/build_manual_pdf.py`	Python 3.12 script	3065 lines	Builds this manual (generated tables, self-test run, LuaLaTeX, bookmark check).

2.3 Veusz and LuaLaTeX (optional)

The `.vszh5` documents are written without Veusz; Veusz is needed only to open them or to render them with `--render` [23]. The renderer finds Veusz on the PATH, in the

usual install folders, or through the environment variable `VEUSZ_EXE` (or `QUICKSYN_VEUSZ`), e.g. `set VEUSZ_EXE=C:\Program Files\Veusz\veusz.exe`. Rebuilding this manual needs `lualatex` from TeX Live or MiKTeX (or `EMF_FFT_LUALATEX` pointing to it); its headings become the PDF bookmarks through hyperref and bookmark [37].

2.4 Spyder

Select the interpreter of the `emf-fft` environment in Spyder (*Tools → Preferences → Python interpreter*; the environment includes `spyder-kernels`) [35]. *Run file* opens the GUI. The script is divided into `# %%` cells (Section 11); run them in order with *Run cell*, then use the scratch cell at the end.

3 Quick start

```
:: 0  the GUI (everything below is also in its menus)
python emf_envelope_fft_v0_0_4.py

:: 1  check the installation; verify every transform and unit conversion
python emf_envelope_fft_v0_0_4.py --selftest
python emf_envelope_fft_v0_0_4.py --verify

:: 2  see how a file will be read (columns, units, time base)
python emf_envelope_fft_v0_0_4.py --inspect export.csv

:: 3  analyse one probe log; also render PDF and PNG with Veusz; plots in mW/cm2
python emf_envelope_fft_v0_0_4.py export.csv --probe EFD-4091 --render --display-unit mW/cm2

:: 4  undo the probe's averaging and force Lomb-Scargle
python emf_envelope_fft_v0_0_4.py export.csv --probe EFD-4091 --correct-probe --lomb on

:: 5  several files in parallel, on the GPU if one is available
python emf_envelope_fft_v0_0_4.py a.csv b.csv c.csv --jobs 3 --gpu

:: 6  SignalShark spectrogram, five strongest carriers, absolute channel power (RBW 100 kHz)
python emf_envelope_fft_v0_0_4.py rec.h5 --source signalshark --rf-channels auto:5 --rbw-hz 1e5

:: 7  EHP-50 TS: the whole folder of auto-saved spectra
python emf_envelope_fft_v0_0_4.py D:\EHP50\2026 --source ehp

:: 8  follow a file that another program is writing (Live tab of the GUI)
python emf_envelope_fft_v0_0_4.py --live growing.csv --probe EF0691 --window-s 120

:: 9  units: spectra in mW/cm2, fields in dBuV/m; Veusz also in W/m2, dBm/m2 and mA/m
python emf_envelope_fft_v0_0_4.py export.csv --display-unit mW/cm2,dBuV/m --time-display "E field"
python emf_envelope_fft_v0_0_4.py export.csv --veusz-units W/m2,dBm/m2,mA/m --render
python emf_envelope_fft_v0_0_4.py --list-units
```

3.1 Typical workflow

In the GUI: add the files (or drag them onto the window), choose the probe, press *Inspect* (Ctrl+I) and check how each file is read, press *Analyse* (F5), and look through the result tabs; *Veusz → Render* gives the PDF. On the command line:

1. Export the measurement from the instrument software as text (CSV/TXT) or Excel, or copy the SignalShark `.h5` file.
2. Run `--inspect` and check the detected delimiter, decimal mark, columns, unit and sample interval. Correct anything wrong with the override options (Section 5.6).

3. Run the analysis with `--probe` set, so that the sample rate, integration time and range limits of the probe are known.
4. Open `<stem>_envelope_fft.vszh5` in Veusz, or use the rendered PDF, whose bookmarks list every page (Section 8).
5. Read `<stem>_summary.json`: peaks, harmonic families with their rpm, and warnings about timing, gaps, aliasing and range.

4 The graphical user interface

4.1 Starting the GUI

Running the script without arguments opens the main window:

```
python emf_envelope_fft_v0_0_4.py
python emf_envelope_fft_v0_0_4.py a.csv rec.h5 D:\EHP50\2026 --gui    (inputs listed)
python emf_envelope_fft_v0_0_4.py --live growing.csv --probe EF0691   (live tab running)
```

In Spyder, *Run file* (F5) opens the same window; with Spyder's Qt graphics backend the window runs inside Spyder's event loop. The GUI is written against a Qt abstraction layer so that one source runs on any binding, with PySide6 as the binding [24, 40, 25]: Qt.py is tried first (as in the QuickSyn controller), then QtPy, then PySide6 directly; the environment variable `EMF_FFT_QT_LAYER=qtpy` puts QtPy first. The layer and binding in use are shown in the log and in *Help → About*. Every plot is drawn with pyqtgraph [26]. The command line, batch processing and the verification do not need Qt.

4.2 Window layout

Figure 1 shows the window:

- **Settings** dock (left): the input list and every analysis, output and display option, in groups (Table 2). Files and folders can be added with the buttons, the File menu, *File → Recent inputs* or by dragging them onto the window; right-click the list for Inspect, Remove and Open containing folder.
- **Toolbar**: add files or folders, Inspect, Analyse, Stop, Verify, Export plot, Open in Veusz, Light/Dark, and the power-density (S) and E-field (E) units.
- **Analysis** tab (centre): a *Result* selector (one entry per probe log or spectrogram channel) above the result tabs of Table 3.
- **Live** tab: following a growing file (Section 9).
- **Log** dock (bottom): time-stamped messages, notes and warnings of every analysis, files written.
- **Status bar**: messages, the data coordinates under the mouse (cursor readout, in the axis units; for images also the pixel value) and a progress bar (files done / files listed).

Analysis, inspection, verification and the self-test run in a background thread (analysis of several files optionally in parallel processes), so the window stays responsive; *Stop* ends a run after the current file. Plots support pyqtgraph's mouse zoom and pan; right-click a plot for its own menu (view all, axis ranges, log axes, grid, export), and *View → Auto-range* (Ctrl+0) shows all data again. Axis labels always state the unit; the automatic SI rescaling of pyqtgraph ("×0.001") is switched off so that values read in the unit named on the axis.

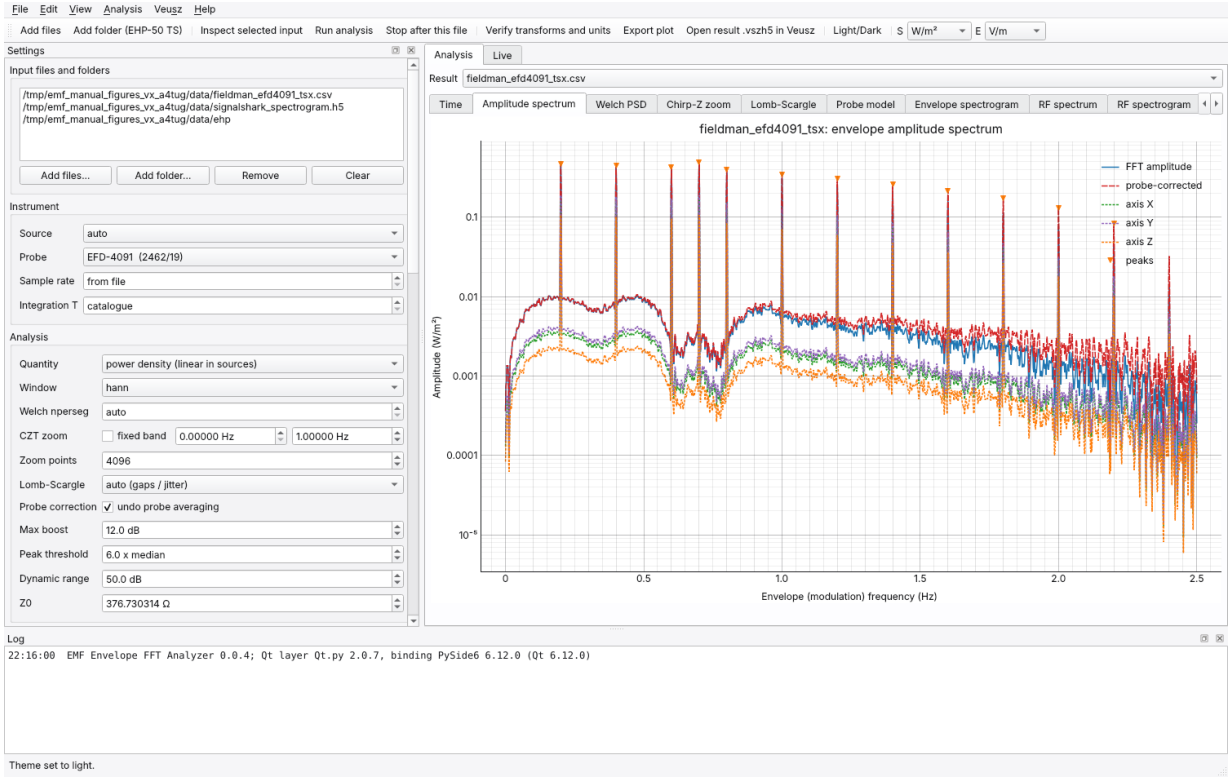


Figure 1: Main window, light theme: settings dock (left), toolbar, result selector and tabs (amplitude spectrum of a FieldMan/EFD-4091 log with the probe-corrected trace, X/Y/Z axes and peak markers), log dock and status bar. *(Synthetic data.)*

Table 2: Settings panel groups and the matching command-line options.

Group	Fields	Command line
Input files	list, Add files, Add folder, Remove, Clear; drag and drop; right-click menu	positional inputs
Instrument	Source, Probe, Sample rate, Integration T	--source, --probe, --fs, --t-int
Analysis	Quantity, Window, Welch nperseg, CZT zoom band and points, Lomb-Scargle, Probe correction, Max boost, Peak threshold, Dynamic range, Z0	--quantity, --window, --nperseg, --zoom, --zoom-points, --lomb, --correct-probe, --max-boost-db, --snr, --dyn-range-db, --z0
Spectrograms	Channels, antenna factor, RBW, ENBW factor	--rf-channels, --antenna-factor-db, --rbw-hz, --enbw-factor
Columns and format	Time column, Value column, Unit, Delimiter, Decimal mark, Day first, HDF5 levels/frequency/time	--time-col, --value-col, --unit, --sep, --decimal, --dayfirst, --h5-level/--h5-freq/--h5-time
Output	write files, folder, render with Veusz, pickle, parallel files, GPU	--out, --render, --pickle, --jobs, --gpu
Units and display	one unit per family (power density, E, H, B, port power, port voltage), Time plot shows, RF levels as, Spectrum y axis (re-plot at once)	--display-unit LIST, --time-display, --rf-level-display
Veusz documents in several units	tick list of every unit, layout (sections or files)	--veusz-units LIST, --veusz-layout

Table 3: Result tabs of the Analysis tab.

Tab	Shows
Time	power density, E or H (plane wave), the logged field or the analysed quantity, in the chosen units; date axis when the instrument clock is known
Amplitude spectrum	FFT amplitude (log, linear or dB), probe-corrected, X/Y/Z axes, peak markers

Table 3 (continued)

Tab	Shows
Welch PSD	PSD in dB re 1 (display unit)^2/Hz
Chirp-Z zoom	dense CZT samples and the FFT bins around the strongest line
Lomb-Scargle	amplitude from the raw time stamps against the FFT
Probe model	20 log10 |sinc(f T)|
Envelope spectrogram	short-time PSD, time x envelope frequency
RF spectrum, RF spectrogram	mean/max-hold spectrum and level image of a spectrogram input in the power-density unit or as plane-wave E or H (RF levels as)
Modulation map	envelope spectrum of every frequency bin (dB rel. max)
Peaks and families	peak table (frequency, amplitude, SNR, CZT values, period) and harmonic families with rpm
Summary	readable summary, files written, and the summary file content (SI)
Inspect	how the selected inputs are read
Verification	the verify_transforms table (Analysis > Verify)

4.3 Units, display and themes

The *Units and display* group of the settings panel (Figure 2) holds one unit selector per quantity family: power density, electric field E, magnetic field H, magnetic flux density B, port power and port voltage (the last two for dBm or dBµV levels without an antenna factor); the same choices are in *View → Units*, and the power-density and E-field units are also on the toolbar. Each family offers linear units and level (dB) units, 34 in all (Table 11, Section 6.3): a result is shown in the unit chosen for the family of its analysed quantity, so a FieldMan log in mW/cm^2 and a SignalShark channel in dBm at the port can be on screen with their own units at the same time. *Time plot shows* selects the power density, E or H of the equivalent plane wave, the logged field, or the analysed quantity (Table 12); when the choice cannot be derived from a record (for example E of a dBm port log) the analysed quantity is shown and the axis says so. *RF levels as* shows the spectrum and spectrogram levels of power-density inputs as power density or as plane-wave E or H. *Spectrum y axis* selects a logarithmic, linear or dB-relative (20 log10) amplitude axis; level units are always drawn on a linear axis. Every change re-plots at once without re-analysing: results are kept in SI units, and the peaks table, the summary and the live plots follow the same choices.

The *Veusz documents in several units* group lists every unit with a tick box; the ticked units are added to the Veusz output besides the display units, either as sections of one document or as one document per unit (Section 8.2). *Veusz → Write .vszh5 again* (Ctrl+Shift+W) re-writes the selected result's document(s) in the current units without analysing again, and renders them when *Render* is ticked.

View → Theme selects System default, Light or Dark (Figure 3); the theme recolours the window and every plot, and is remembered.

4.4 Menus and keyboard shortcuts

Table 4 lists every menu command, read from the running window when this manual was built; *Help → Keyboard shortcuts* shows the shortcuts and *Help → Probe catalogue* the probe table with its derived limits. Settings, window layout, theme, last folder and the recent inputs are kept between sessions (QSettings); *File → Save settings* and *Load settings* store and restore the options as JSON, and *Reset settings to defaults* restores the defaults.

Table 4: Menu commands of the main window (read from the running window).

Menu	Command	Shortcut
File	Add files...	`Ctrl+O`
File	Add folder (EHP-50 TS)...	`Ctrl+Shift+O`
File	Clear inputs	
File	Export current plot (PNG, SVG, CSV)...	`Ctrl+E`

Table 4 (continued)

Menu	Command	Shortcut
File	Export peaks table (CSV)...	
File	Save log...	
File	Open output folder	
File	Load settings...	
File	Save settings...	`Ctrl+S`
File	Reset settings to defaults	
File	Quit	
Edit	Copy summary	`Ctrl+Shift+S`
Edit	Copy peaks table	`Ctrl+Shift+P`
Edit	Copy log	
Edit	Clear log	
Edit	Clear results	
Edit	Free memory (results, live buffer, GC)	
View > Theme	System default	
View > Theme	Light	
View > Theme	Dark	
View > Units > Power density	one of: W/m², mW/m², µW/m², mW/cm², µW/cm², nW/cm², dBW/m², dBm/m²	
View > Units > Electric field E	one of: V/m, mV/m, µV/m, kV/m, dBV/m, dBµV/m	
View > Units > Magnetic field H	one of: A/m, mA/m, µA/m, dBµA/m	
View > Units > Magnetic flux density B	one of: T, mT, µT, nT, pT	
View > Units > Port power	one of: W, mW, µW, nW, dBW, dBm	
View > Units > Port voltage	one of: V, mV, µV, dBV, dBµV	
View > Time plot shows	one of: power density, E field, H field, logged field, analysed quantity	
View > RF levels as	power density	
View > RF levels as	E field	
View > RF levels as	H field	
View > Spectrum y axis	log	
View > Spectrum y axis	linear	
View > Spectrum y axis	dB rel. max (20 log10)	
View	Auto-range current plot	`Ctrl+0`
View	Toolbar	
View	Settings panel	`Ctrl+Shift+E`
View	Log panel	`Ctrl+Shift+L`
View	Full screen	`F11`
Analysis	Inspect selected input	`Ctrl+I`
Analysis	Run analysis	`F5`
Analysis	Stop after this file	`Esc`
Analysis	Verify transforms and units	`Ctrl+Shift+V`
Analysis	Run self-test	
Analysis	Live mode	`Ctrl+L`
Veusz	Open result .vszh5 in Veusz	`Ctrl+Shift+Z`
Veusz	Write .vszh5 again (current units)	`Ctrl+Shift+W`
Veusz	Render result to PDF/PNG	
Veusz	Open rendered PDF	
Help	User manual (PDF)	`F1`
Help	Keyboard shortcuts	
Help	Probe catalogue	
Help	About EMF Envelope FFT Analyzer	
Help	About Qt	

4.5 About dialog

Help → About (Figure 4) shows the program name and version, the author, company and e-mail address, the licence (GNU General Public License v3.0 or later) with its warranty notice and a link to the licence text, and under *Show Details* the versions of Python, the Qt layer and binding, pyqtgraph, NumPy, SciPy, pandas, h5py and the Veusz writer.

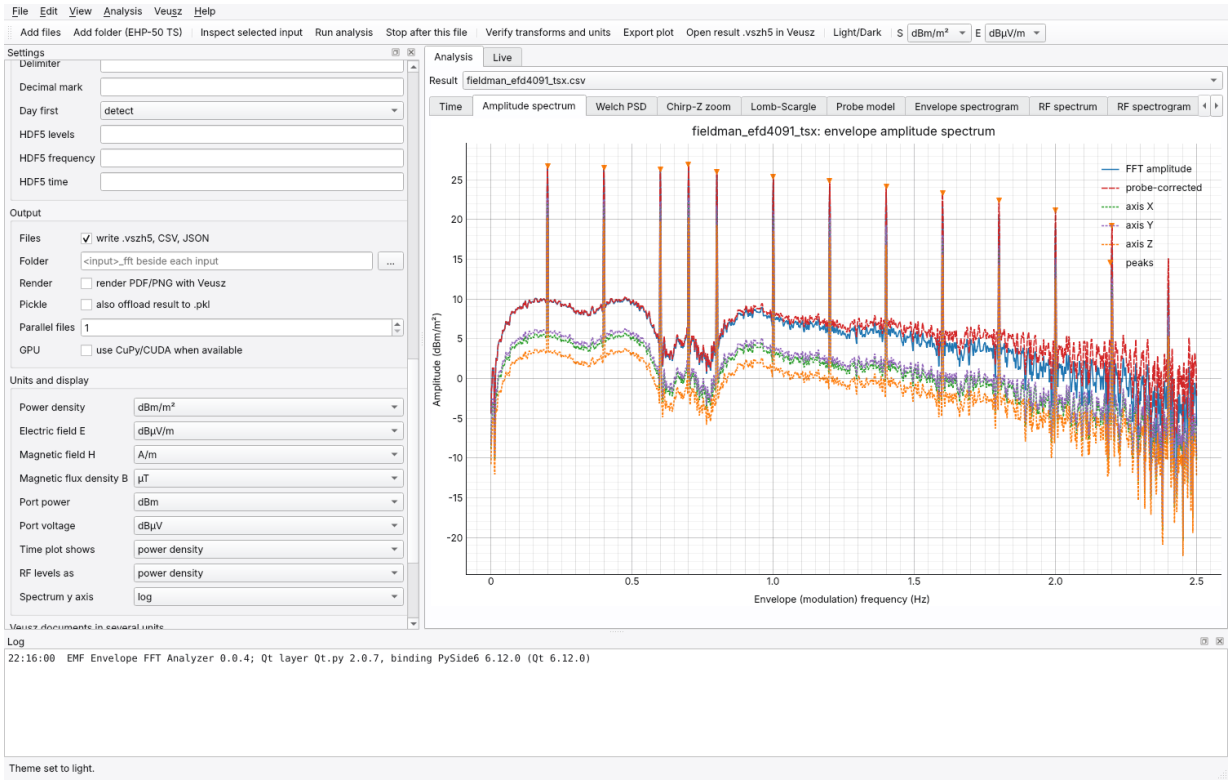


Figure 2: Units and display group (left, scrolled into view): one unit per quantity family, time-plot and RF-level modes; the amplitude spectrum in dBm/m^2 (a level unit, so a linear axis) and the toolbar S and E unit selectors. *(Synthetic data.)*

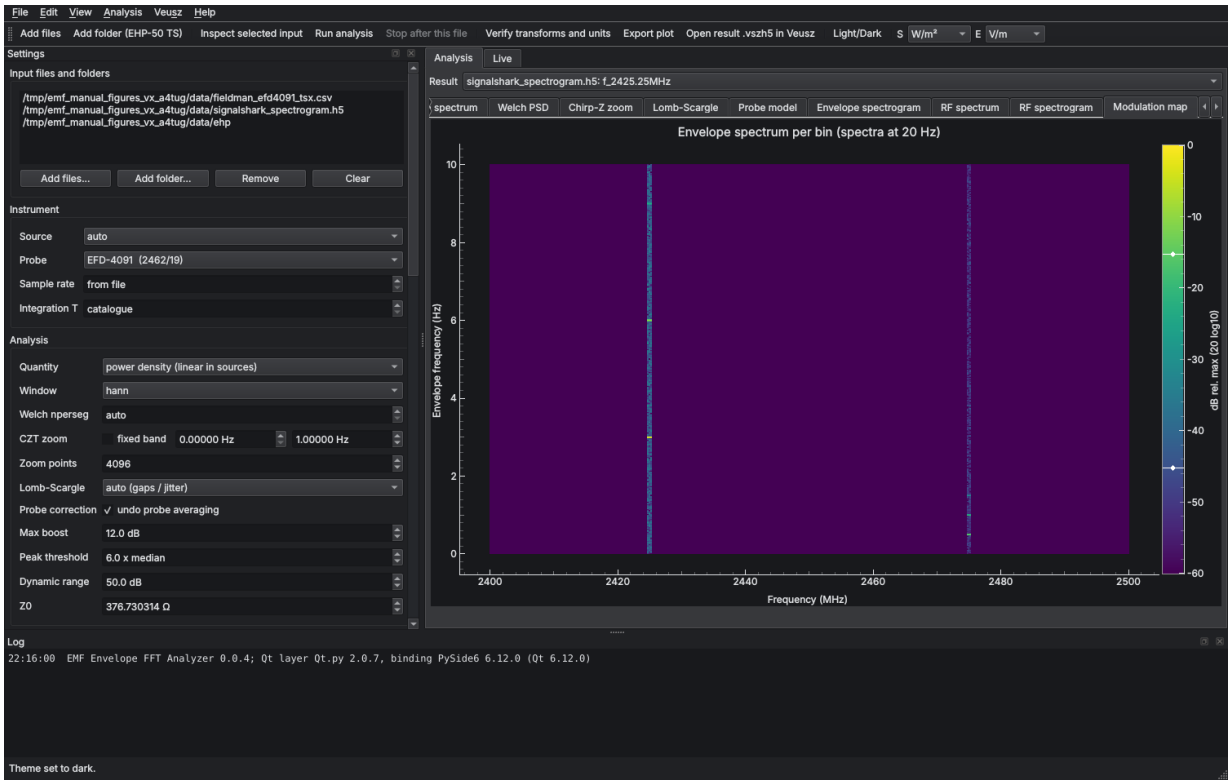


Figure 3: Dark theme: modulation map of a SignalShark spectrogram channel. *(Synthetic data.)*

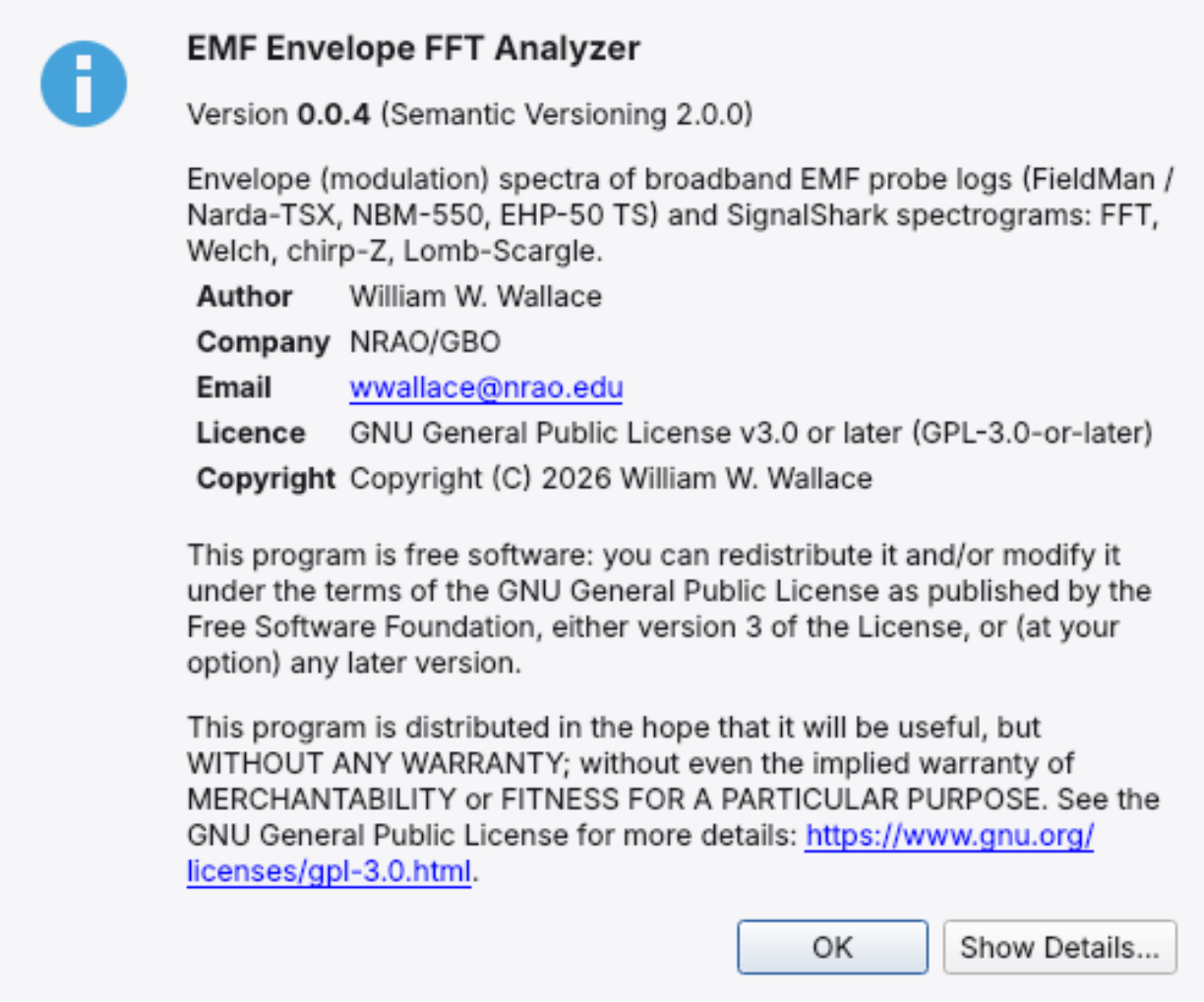


Figure 4: Help → About: author, version and the GPL-3.0-or-later licence. *(Synthetic data.)*

5 Instruments and input files

5.1 Probe catalogue

Table 5 lists the probes the program knows, from the datasheets. `--probe` accepts the names in the first column (case and spaces ignored) and the aliases `EFD-4091E` (treated as the EFD-4091; no separate datasheet for an "E" variant was found), `EF-0691`, `EFD-1891FAST` and the forms without hyphens. The last four columns are derived: the Nyquist frequency $f_s/2$, the response of the probe's averaging at that frequency, and the first null $1/T$ of that response (Section 6.4).

Table 5: Probe catalogue (datasheet values) and the derived envelope limits.

Probe	RF range	f_s (Hz)	T (ms)	ν_N (Hz)	$\lvert H(\nu_N)\rvert$ (dB)	$1/T$ (Hz)	True RMS to (V/m)	Axes	Ref.
EFD-4091 2462/19	2 MHz -- 40 GHz	5	265	2.5	-7.6	3.774	65	rss	[1]
EFD-0691 2462/14B	100 kHz -- 6 GHz	5	265	2.5	-7.6	3.774	17	xyz	[2]
EF 0691 (NBM) NBM probe	100 kHz -- 6 GHz	5	n/p (1/fs used)	2.5	-3.9	5	22	rss	[6]
EFD-9091 2462/18	100 MHz -- 90 GHz	5	275	2.5	-8.3	3.636	61.4	rss	[3]
EFD-1891 Fast 2462/23	2 MHz -- 18 GHz	900	2.3	450	-29.4	434.8	–	rss	[4]

5.2 Readers

The reader is chosen from the file (Table 6); `--source` forces one.

Table 6: Readers and how the input is recognised.

`--source`	Instrument / software	Files	Auto-detected when	Result
`fieldman`, `tsx`	FieldMan or Narda-TSX with EFD-4091, -0691, -9091, -1891	.csv .txt	default for text files	time series (+ X/Y/Z)
`nbm`	NBM-550 / NBM-TS with EF 0691	.csv .txt	a column named 'Result ...'	time series
`xlsx`	any of the above saved as Excel	.xlsx .xls	by extension	time series
`ehp`	EHP-50 TS with EHP-50F (1 Hz - 400 kHz)	folder, `hh_mm_ss_cc.TXT`	a folder, or the EHP file name	spectrogram (folder), spectrum, or time series (logger file)
`signalshark`	SignalShark	.h5 .hdf5; converter .csv	.h5 extension; 8+ numeric column names or a frequency column	spectrogram (or level time series)

5.3 How text exports are read

Narda's software is German-made, so semicolon-separated files with a decimal comma are expected as often as comma-separated files with a decimal point.

1. **Encoding**: UTF-8 (with or without byte-order mark), then Windows-1252, then Latin-1.
2. **Delimiter**: the first of tab, `;`, `,`, `|` that occurs the same number of times on the last 50 non-blank lines; whitespace otherwise. Semicolon is tried before comma because a decimal comma puts a comma inside every number.
3. **Decimal mark**: comma when numbers such as `1,23` appear and the delimiter is not a comma.
4. **Header row**: the first row with the same number of fields as the row below it and more text fields than that row, whose fields are mostly numbers or date/times. Lines above it are kept as metadata and searched for a unit.
5. **Columns**: chosen with the keyword lists of Table 7; the unit comes from `--unit`, the value column's header or the metadata, in that order.
6. **Time**: numbers are seconds (milliseconds when the header says `ms`); anything else is parsed as date/time, day first when dates look like `dd.mm.yyyy`. A date column and a clock column are joined. The first value is the origin of every chunk of a large file, so the time axis is continuous.
7. **Size**: the file is read in chunks of 500 000 rows; columns longer than 20 000 000 samples move to a temporary file on disk (Section 10).

Table 7: Keywords that select columns (case ignored).

Use	Keywords	Note
time column	`time`, `zeit`, `uhrzeit`, `timestamp`, `date`, `datum`, `elapsed`, `t [s]`, `t(s)`, `t/s`, `sec`, `hh:mm`	a date column and a clock column are joined; otherwise the first match
value column, preferred (+5)	`actual`, `act`, `rss`, `total`, `result 1`, `isotrop`, `value`, `wert`, `e-field`, `e field`, `field`, `level`	+3 more if the header names a unit; leftmost column wins ties
value column, avoided (-6)	`max`, `min`, `avg`, `mean`, `average`, `batt`, `temp`, `humid`, `press`, `lat`, `lon`, `alt`, `gps`, `index`, `no.`, `nr`, `flag`, `dist`, `comment`	statistics and house-keeping columns
axis columns	`X`, `Y`, `Z`, `E_x`, `H y`, `field_z`, ...	used for per-axis spectra; their RSS is the value when no value column exists

5.4 EHP-50 TS spectra

EHP-50 TS saves one spectrum per file as `YYYY/MM/DD/hh_mm_ss_cc.TXT` (cc = hundredths of a second) [7]. Given a folder, the program reads every `.txt`/`.csv` below it, takes the time of each spectrum from its path (the file date when the name does not match), sorts them, and puts every spectrum on the frequency axis of the first one (linear interpolation when they differ). The total or RSS column is preferred over X, Y, Z. The result is analysed as a spectrogram (Section 6.15). Note that EHP-50 TS records only the EHP-50F (1 Hz to 400 kHz); it cannot record the RF probes.

5.5 SignalShark files

The layout of SignalShark's `.h5` files is not published. The reader takes the largest two-dimensional numeric dataset as levels (time × frequency, transposed if the axes say so), a one-dimensional dataset of matching length whose name or values look like frequencies as the frequency axis, and one that increases as the time axis; otherwise start/stop/centre/span and time-resolution attributes. `--h5-level`, `--h5-freq` and `--h5-time` name the datasets explicitly; `--inspect` prints the whole HDF5 tree. CSV files made with Narda's NSL *Converter* script [8] are read in wide format (one row per spectrum, the header holding the frequencies), long format (time, frequency, level columns) or transposed wide format (one row per frequency). Levels in dBm or dBµV are converted to field strength only when an antenna factor is given (`--antenna-factor-db`).

5.6 Overriding the detection

`--source`, `--time-col` (`"Date+Time"` joins two columns), `--value-col`, `--unit`, `--sep`, `--decimal`, `--dayfirst` and `--fs` (for files without a time column) override the corresponding guess. Column names or zero-based indices are accepted.

6 Methods: theory and implementation

This section describes every method the program uses: the physics of the measurement, the unit conversions, and each time-to-frequency transform (FFT amplitude spectrum, Welch power spectral density, chirp-Z zoom, Lomb–Scargle periodogram, short-time FFT), followed by the probe model, peak finding, harmonic families and the spectrogram methods. For each transform it gives the idea, the derivation, the algorithm as implemented, the parameters and their trade-offs, and how its correctness was verified (Section 12). Table 8 is an overview.

Table 8: The methods at a glance.

Method	Question it answers	Output (unit)	Resolution	Used
FFT amplitude	how strong is each modulation frequency?	amplitude (W/m2)	fs/N	always
Welch PSD	how is envelope power spread? noise floor	PSD ((W/m2)^2/Hz)	ENBW fs/L	always
Chirp-Z zoom	exact frequency and amplitude of a line	amplitude (W/m2)	fs/N (dense samples)	around the strongest line, or --zoom
Lomb-Scargle	spectrum without resampling irregular stamps	amplitude (W/m2)	1/T_span	gaps or jitter, or --lomb on
Short-time FFT	does a modulation come and go?	PSD vs time	fs/L and L/fs	long logs
Probe model	how much does the probe's averaging attenuate?	|H| (20 log10)	–	probe or T known
Per-bin modulation	which RF carrier is modulated at which rate?	dB rel. max	spectra rate / N_t	spectrograms

6.1 Square-law detection adds powers

A diode detector operated in its square-law region delivers a voltage proportional to the square of the RF voltage. Let the field at the probe be the sum of M sources,

$$E(t) = \sum_{i=1}^{M} E_i \cos(2\pi f_i t + \varphi_i). \tag{6.1}$$

The detector output, averaged by a video filter that is much slower than any difference frequency $|f_i - f_j|$, is

$$\begin{aligned} \overline{E^2} &= \sum_i \sum_j E_i E_j \, \overline{\cos(2\pi f_i t + \varphi_i)\cos(2\pi f_j t + \varphi_j)} \\ &= \sum_i \sum_j \frac{E_i E_j}{2}\left[\overline{\cos\left(2\pi(f_i - f_j)t + \varphi_i - \varphi_j\right)} + \overline{\cos\left(2\pi(f_i + f_j)t + \varphi_i + \varphi_j\right)}\right] \\ &= \sum_i \frac{E_i^2}{2} = \sum_i E_{i,\mathrm{rms}}^2, \end{aligned} \tag{6.2}$$

because every term with $i \neq j$ and every sum-frequency term averages to zero. Three orthogonal sensors add in the same way, $E^2 = E_x^2 + E_y^2 + E_z^2$ (the isotropic RSS value). With the probe's

frequency response $r(f)$ and the plane-wave power density $S = E_{\mathrm{rms}}^2/Z_0$, the displayed value is

$$S_{\mathrm{disp}}(t) = \sum_i r(f_i)\, S_i(t), \qquad E_{\mathrm{disp}}(t) = \sqrt{Z_0\, S_{\mathrm{disp}}(t)}. \tag{6.3}$$

Equation (6.2) holds only while every diode stays in its square-law region, which for the EFD probes is the True-RMS range of Table 5; above it the program warns.

6.2 Why the transform is taken of power

The Fourier transform is linear, so applying it to (6.3) gives

$$\mathcal{F}\{S_{\mathrm{disp}}\}(\nu) = \sum_i r(f_i)\, \mathcal{F}\{S_i\}(\nu)\,: \tag{6.4}$$

the envelope spectrum of the log is the weighted sum of the envelope spectra of the sources, and each source's modulation lines appear unchanged. The field $E_{\mathrm{disp}} = \sqrt{Z_0 S_{\mathrm{disp}}}$ does not have this property. Take two sources $S_1 = a + b\cos 2\pi\nu_1 t$ and $S_2 = c + d\cos 2\pi\nu_2 t$ with $A = a + c$ and $x(t) = b\cos 2\pi\nu_1 t + d\cos 2\pi\nu_2 t$, $|x| < A$. The Taylor series of the square root gives

$$\sqrt{A + x} = \sqrt{A}\left(1 + \frac{x}{2A} - \frac{x^2}{8A^2} + \ldots\right), \tag{6.5}$$

$$\begin{aligned} x^2 = {} & \frac{b^2 + d^2}{2} + \frac{b^2}{2}\cos 4\pi\nu_1 t + \frac{d^2}{2}\cos 4\pi\nu_2 t \\ & + bd\,[\cos 2\pi(\nu_1 - \nu_2)t + \cos 2\pi(\nu_1 + \nu_2)t], \end{aligned} \tag{6.6}$$

so the spectrum of E contains lines at $2\nu_1$, $2\nu_2$ and $\nu_1 \pm \nu_2$ that no source produces (intermodulation). The program therefore analyses power density (W/m^2) by default (`--quantity power`); `field` and `db` are available for comparison only.

6.3 Units, conversions and dB conventions

Electric and magnetic field. For a plane wave in free space the Poynting vector has the magnitude $S = E\,H$ and the fields are in the ratio $E/H = Z_0 = \mu_0 c$, so

$$S = EH = \frac{E^2}{Z_0} = Z_0 H^2, \qquad E = \sqrt{S Z_0}, \qquad H = \sqrt{S/Z_0}. \tag{6.7}$$

The program uses $Z_0 = 376.730313668\,\Omega$ (from the CODATA 2018 μ_0); the CODATA 2022 value is $376.730\,313\,412(59)\,\Omega$ [32], a difference of 7×10^{-10}. Narda's displays use round conversions near $377\,\Omega$; `--z0 377` reproduces them. Readings of E, H or a power density are converted to S in W/m^2 for the analysis; magnetic flux density is analysed as B^2 (proportional to the magnetic energy density), and EHP-50F fields as the field squared, because at 1 Hz–400 kHz the fields are quasi-static and no plane-wave power density exists. Equation (6.7) holds only in the far field; the conversions of the time plots below rely on it.

Table 9: Power-density input units: W/m^2 per unit (reading × factor = W/m^2).

Key	W/m^2 per unit	Label
`W/m2`	1	W/m²
`mW/m2`	0.001	mW/m²
`uW/m2`	1e-06	µW/m²
`mW/cm2`	10	mW/cm²
`uW/cm2`	0.01	µW/cm²
`nW/cm2`	1e-05	nW/cm²

Logarithmic input units. From dBµV/m: $E = 10^{(L-120)/20}\,\mathrm{V/m}$, because $L = 20\log_{10}(E/1\,\mu\mathrm{V/m})$ and $1\,\mu\mathrm{V} = 10^{-6}\,\mathrm{V}$. From dBm at a 50 Ω port, $P = V^2/R$ gives

$$\begin{aligned} L_{\mathrm{dB\mu V}} &= 20\log_{10}\frac{V}{1\,\mu\mathrm{V}} = 10\log_{10}\frac{PR}{10^{-12}\,\mathrm{V}^2} \\ &= 10\log_{10}\frac{P}{1\,\mathrm{mW}} + 10\log_{10}\frac{10^{-3}\cdot 50}{10^{-12}} = L_{\mathrm{dBm}} + 106.9897, \end{aligned} \tag{6.8}$$

which the program uses exactly (version 0.0.2 used the rounded 107 dB). With the antenna factor $AF = E/V$ in dB(1/m), $L_{\mathrm{dB\mu V/m}} = L_{\mathrm{dB\mu V}} + AF$; without `--antenna-factor-db`, dBm levels are analysed as mW at the analyzer port and dBµV levels as $\mu\mathrm{V}^2$. A power density in dBm/m² is $10^{L/10}\,\mathrm{mW/m^2}$.

Display units: families and the two rules. Every analysed quantity belongs to a *family* (Table 10): power density (base unit W/m²), electric field (V/m), magnetic field (A/m), flux density (T), port power (mW) and port voltage (µV). One display unit is chosen per family (`--display-unit mW/cm2,dBuV/m`: each unit sets its own family, the others keep their defaults). A value x held in the family's base unit is shown in a *linear* unit as $k\,x$, where k is the number of display units per base unit (Table 11); for example $1\,\mathrm{W/m^2} = 10^{-3}\,\mathrm{W}/10^4\,\mathrm{cm^2}\cdot 10^3\,\mathrm{mW/W} = 0.1\,\mathrm{mW/cm^2}$, so $k = 0.1$. In a *level* unit it is shown as

$$L = D\,\log_{10}(k\,x)\ \mathrm{dB}, \qquad D = \begin{cases} 10 & \text{power quantities (power density, port power),} \\ 20 & \text{root-power quantities } (E,\ H,\ \text{port voltage}), \end{cases} \tag{6.9}$$

the convention of IEC 60027-3 and ISO 80000-1 [41, 42]: the square of a root-power quantity is proportional to power, so both give the same level change for the same change of power. Hence $1\,\mathrm{W/m^2} = 0\,\mathrm{dBW/m^2} = 30\,\mathrm{dBm/m^2}$ ($k = 10^3$), $1\,\mathrm{V/m} = 120\,\mathrm{dB\mu V/m}$ ($k = 10^6$, $D = 20$), $1\,\mathrm{mW} = 0\,\mathrm{dBm} = -30\,\mathrm{dBW}$. A factor ten in x changes L by 10 dB for power and 20 dB for root-power quantities. Values at or below zero have no level; they are clipped at 10^{-15} of the largest value of the trace (NaN stays NaN).

Table 10: Analysed quantities, their origin and their unit family (`QUANTITY_FAMILY`); the last column is the default display unit.

Analysed	Comes from	Family	Default
`W/m^2`	power density: S logged, E^2/Z0, Z0 H^2, dBuV/m, or dBm/dBuV with an antenna factor	Power density	W/m²
`V/m`	E with --quantity field (E logged or sqrt(S Z0))	Electric field E	V/m
`A/m`	H with --quantity field	Magnetic field H	A/m
`T`	B with --quantity field	Magnetic flux density B	µT
`mW`	dBm at the analyzer port, no antenna factor	Port power (no antenna factor)	dBm
`uV^2`	dBuV at the analyzer port, no antenna factor	Port voltage (no antenna factor), squared	dB re 1 µV²
`T^2`	B logged, analysed as B^2 (power-like)	Magnetic flux density B, squared	µT²
`(V/m)^2`	EHP-50 TS E field squared (quasi-static, no Z0)	Electric field E, squared	(V/m)²
`(A/m)^2`	EHP-50 TS H field squared	Magnetic field H, squared	(A/m)²
`(T)^2`	EHP-50 TS B field squared	Magnetic flux density B, squared	µT²
%, dB rel. max, a.u.	% of standard, --quantity db, unknown unit	none: shown as analysed	–

Squared quantities. A field squared (E^2 of the EHP-50 TS, B^2, $\mu\mathrm{V}^2$) is shown in the square of the unit chosen for its root family: with factor k for the root, $k^2x^2 = (kx)^2$, e.g. µT → $\mu\mathrm{T}^2$ ($k^2 = 10^{12}$). For a root level unit, the square is a power level of the same number:

$$10\log_{10}\left(k^2x^2\right) = 20\log_{10}(kx), \tag{6.10}$$

labelled "dB re 1 $(\mu\mathrm{V/m})^2$" for dBµV/m.

Spectra in display units. The transforms are linear, so for a linear unit every amplitude (FFT, chirp-Z, Lomb–Scargle, the peaks) is multiplied by k, and a power spectral density by k^2:

$$\hat{P}_{\text{display}}(\nu) = k^2\,\hat{P}(\nu) \quad \Rightarrow \quad 10\log_{10}\hat{P}_{\text{display}} = 10\log_{10}\hat{P} + 20\log_{10}k. \tag{6.11}$$

The PSD is always drawn as $10\log_{10}$ in dB re 1 (unit)^2/Hz; for a level unit the linear reference is used (dBm/m^2 → (mW/m^2)^2/Hz, +60 dB against (W/m^2)^2/Hz; mW/cm^2 → −20 dB). Amplitudes in a level unit follow Eq. (6.9) bin by bin; their axis is linear.

Time plots: power density, E and H. The time plot (and the time pages of the Veusz output) shows the record as power density, E or H of the equivalent plane wave, using Eq. (6.7) point by point. In level units,

$$20\log_{10}\frac{E}{1\ \mu\text{V/m}} = 10\log_{10}\frac{S}{1\ \text{W/m}^2} + 10\log_{10}\frac{Z_0}{1\ \Omega} + 120, \tag{6.12}$$

$$20\log_{10}\frac{H}{1\ \mu\text{A/m}} = 10\log_{10}\frac{S}{1\ \text{W/m}^2} - 10\log_{10}\frac{Z_0}{1\ \Omega} + 120, \tag{6.13}$$

with $10\log_{10} Z_0 = 25.7603$ dB: 1 W/m^2 is 145.76 dBμV/m and 94.24 dBμA/m, and 1 mW/cm^2 (S = 10 W/m^2) is E = 61.378 V/m = 155.76 dBμV/m and H = 162.92 mA/m. B is shown only when B was logged: for low-frequency magnetic fields no free-space conversion is assumed.

RF levels. Spectrum and spectrogram levels are $\ell = 10\log_{10} x$ of the bin power in its base unit. In the display unit they are $\ell + 10\log_{10} k$ (label "dB re 1 mW/cm^2" for a linear unit, "dBm/m^2" for a level unit). For power-density inputs they can also be shown as the plane-wave field, $\ell + 10\log_{10} Z_0 + 20\log_{10} k_E$ for E and $\ell - 10\log_{10} Z_0 + 20\log_{10} k_H$ for H, which is the field level of the power in the bin.

Table 11: Display units (`DISPLAY_UNITS`, also `--list-units`): the value shown for a value held in the family's base unit, and the level rule.

Family	Key	Label	Shown value	Rule
Power density	`W/m2`	W/m²	1 x value in W/m²	linear
Power density	`mW/m2`	mW/m²	1000 x value in W/m²	linear
Power density	`uW/m2`	μW/m²	1e+06 x value in W/m²	linear
Power density	`mW/cm2`	mW/cm²	0.1 x value in W/m²	linear
Power density	`uW/cm2`	μW/cm²	100 x value in W/m²	linear
Power density	`nW/cm2`	nW/cm²	100000 x value in W/m²	linear
Power density	`dBW/m2`	dBW/m²	10 log10(1 x value in W/m²)	10 log10 (power level re 1 W/m²)
Power density	`dBm/m2`	dBm/m²	10 log10(1000 x value in W/m²)	10 log10 (power level re 1 mW/m²)
Electric field E	`V/m`	V/m	1 x value in V/m	linear
Electric field E	`mV/m`	mV/m	1000 x value in V/m	linear
Electric field E	`uV/m`	μV/m	1e+06 x value in V/m	linear
Electric field E	`kV/m`	kV/m	0.001 x value in V/m	linear
Electric field E	`dBV/m`	dBV/m	20 log10(1 x value in V/m)	20 log10 (root-power level re 1 V/m)
Electric field E	`dBuV/m`	dBμV/m	20 log10(1e+06 x value in V/m)	20 log10 (root-power level re 1 μV/m)
Magnetic field H	`A/m`	A/m	1 x value in A/m	linear
Magnetic field H	`mA/m`	mA/m	1000 x value in A/m	linear
Magnetic field H	`uA/m`	μA/m	1e+06 x value in A/m	linear
Magnetic field H	`dBuA/m`	dBμA/m	20 log10(1e+06 x value in A/m)	20 log10 (root-power level re 1 μA/m)
Magnetic flux density B	`T`	T	1 x value in T	linear

Table 11 (continued)

Family	Key	Label	Shown value	Rule
Magnetic flux density B	`mT`	mT	1000 x value in T	linear
Magnetic flux density B	`uT`	µT	1e+06 x value in T	linear
Magnetic flux density B	`nT`	nT	1e+09 x value in T	linear
Magnetic flux density B	`pT`	pT	1e+12 x value in T	linear
Port power (no antenna factor)	`W`	W	0.001 x value in mW	linear
Port power (no antenna factor)	`mW`	mW	1 x value in mW	linear
Port power (no antenna factor)	`uW`	µW	1000 x value in mW	linear
Port power (no antenna factor)	`nW`	nW	1e+06 x value in mW	linear
Port power (no antenna factor)	`dBW`	dBW	10 log10(0.001 x value in mW)	10 log10 (power level re 1 W)
Port power (no antenna factor)	`dBm`	dBm	10 log10(1 x value in mW)	10 log10 (power level re 1 mW)
Port voltage (no antenna factor)	`V`	V	1e-06 x value in µV	linear
Port voltage (no antenna factor)	`mV`	mV	0.001 x value in µV	linear
Port voltage (no antenna factor)	`uV`	µV	1 x value in µV	linear
Port voltage (no antenna factor)	`dBV`	dBV	20 log10(1e-06 x value in µV)	20 log10 (root-power level re 1 V)
Port voltage (no antenna factor)	`dBuV`	dBµV	20 log10(1 x value in µV)	20 log10 (root-power level re 1 µV)

Table 12: Time-plot modes (`--time-display`) and RF-level modes (`--rf-level-display`).

Setting	Mode	Shows
Time plot	`power density`	S in the power-density unit; from a logged E or H through S = E^2/Z0 = Z0 H^2
Time plot	`E field`	E in the E-field unit; logged, or E = sqrt(S Z0) = Z0 H of the plane wave
Time plot	`H field`	H in the H-field unit; logged, or H = sqrt(S/Z0) = E/Z0 of the plane wave
Time plot	`logged field`	the field as logged (E, H or B) in its family's unit
Time plot	`analysed quantity`	the transformed quantity in its family's unit
RF levels	`power density`	levels of power-density inputs in dB re 1 (power-density unit), or the level unit
RF levels	`E field`	20 log10 of the plane-wave E: level + 10 log10(Z0) + 20 log10(k)
RF levels	`H field`	20 log10 of the plane-wave H: level - 10 log10(Z0) + 20 log10(k)

Two dB conventions in the envelope analysis. RF *levels* are physical powers and follow Eq. (6.9). The envelope analysis treats the record $S(t)$ as the *signal* of a signal-processing chain; as is usual for any signal, its Fourier amplitudes relative to their maximum and the filter response $|H|$ are expressed as $20\log_{10}$ of an amplitude ratio ("dB rel. max ($20\log10$)", "$20\log10\,|H|$"), and its power spectral density, already a squared quantity, as $10\log_{10}$. Absolute amplitude spectra in a level unit (dBm/m^2, dBµV/m) use the rule of their unit, Eq. (6.9). Every dB axis of the program states which convention it uses.

6.4 Sampling, probe averaging and aliasing

A log sampled at f_s holds envelope content up to the Nyquist frequency $\nu_N = f_s/2$, with resolution $\Delta\nu = f_s/N = 1/T_{\mathrm{rec}}$. Each sample is an average over the integration time T. Modelling that

average as a moving (boxcar) average,

$$y(t) = \frac{1}{T}\int_{t-T}^{t} x(\tau)\,d\tau = (h * x)(t), \qquad h(t) = \begin{cases} 1/T, & 0 \le t < T \\ 0, & \text{otherwise,} \end{cases} \tag{6.14}$$

its frequency response is

$$\begin{aligned} H(\nu) &= \int_0^T \frac{1}{T} e^{-i2\pi\nu t}\,dt = \frac{1 - e^{-i2\pi\nu T}}{i2\pi\nu T} = e^{-i\pi\nu T}\,\frac{e^{i\pi\nu T} - e^{-i\pi\nu T}}{2i\,\pi\nu T} \\ &= e^{-i\pi\nu T}\,\frac{\sin \pi\nu T}{\pi\nu T} = e^{-i\pi\nu T}\,\mathrm{sinc}(\nu T), \qquad |H(\nu)| = |\mathrm{sinc}(\nu T)|. \end{aligned} \tag{6.15}$$

For the EFD-4091 ($T = 265$ ms) this gives -2.39 dB at 1.5 Hz and -7.55 dB at the Nyquist frequency 2.5 Hz, with the first null at $1/T = 3.774$ Hz. Sampling folds any content at ν to

$$\nu_{\text{alias}} = \left|\nu - k f_s\right|, \qquad k = \mathrm{round}(\nu / f_s), \tag{6.16}$$

attenuated only by $|H(\nu)|$: a 3 Hz envelope line at $f_s = 5$ Hz appears at 2 Hz, -12.4 dB down. The boxcar is a model; Narda publishes only a nominal integration time. The verification confirms that $|\mathrm{sinc}(\nu T)|$ is the response of a moving average (closed form at 1 MHz sampling, error $< 10^{-11}$) and that a simulated 265 ms average of a 1.5 Hz modulation is attenuated by exactly $|\mathrm{sinc}|$. Figure 10 shows the model.

6.5 Putting the log on a uniform grid

The FFT, Welch and the short-time FFT need equally spaced samples. The sample interval is the mean of the intervals that are not gaps (an interval longer than 1.6 times the median is a gap). The mean is used rather than the median because time stamps rounded to a microsecond make the median of a 900 Hz log 0.001111 s, and a grid 0.01 % off drifts in phase against the data, which adds side lines and loses amplitude. When the stamps are within 1 % of uniform (median $|\Delta t - \overline{\Delta t}|/\overline{\Delta t} < 0.01$), with no gaps and the expected count, the samples are used as they are; otherwise they are linearly interpolated onto the grid and gaps are bridged. The gap-free blocks are recorded for Welch, and the raw stamps are kept for Lomb–Scargle.

6.6 Windows

A record of finite length is implicitly multiplied by a rectangle, whose transform has high side lobes: a strong line leaks into every other bin. A smooth window $w[n]$ that falls to zero at the record ends lowers the side lobes at the cost of a wider main lobe. Four numbers describe a window for this program (Table 13, computed by the build tool from the SciPy windows actually used):

- the *coherent gain* $\mathrm{CG} = \sum_n w[n]/N$, which the amplitude scaling divides out (Section 6.7);
- the *equivalent noise bandwidth* $\mathrm{ENBW} = N\sum_n w[n]^2/(\sum_n w[n])^2$ in bins, the width of the rectangular filter that passes the same noise power; it sets the resolution of Welch and the noise floor of the amplitude spectrum;
- the *scalloping loss*, the amplitude lost by a line half a bin from a bin centre, $|W(\tfrac{1}{2})|/|W(0)|$; the chirp-Z zoom removes it;
- the *overlap correlation* at 50 % overlap, $c = \sum_n w[n]\,w[n + L/2]/\sum_n w[n]^2$ [16], which limits how much overlapping Welch segments reduce the variance (Section 6.8).

Hann (the default) is a good general choice; flattop reads amplitudes between bins almost without scalloping, at the price of a wide main lobe; boxcar (rectangular) gives the finest resolution and exact Parseval sums but leaks; Blackman-Harris suppresses side lobes for a high dynamic range.

Table 13: Window properties computed from the SciPy windows (N = 4096, periodic): coherent gain, equivalent noise bandwidth, scalloping loss at half a bin, highest side lobe, overlap correlation at 50 % and Welch variance factor $1 + 2c^2$.

Window	CG	ENBW (bins)	Scalloping (dB)	Side lobe (dB)	Overlap c	$1 + 2c^2$
`boxcar`	1.000	1.000	-3.92	-13.3	0.500	1.500
`hann`	0.500	1.500	-1.42	-31.5	0.167	1.056
`hamming`	0.540	1.363	-1.75	-42.7	0.234	1.109
`blackman`	0.420	1.727	-1.10	-58.1	0.090	1.016
`blackmanharris`	0.359	2.004	-0.83	-92.0	0.038	1.003
`flattop`	0.216	3.770	-0.01	0.0	-0.015	1.000

6.7 FFT amplitude spectrum

Purpose. The amplitude of every envelope frequency: a power-density modulation $S(t) = \bar{S} + A\cos(2\pi\nu_0 t + \varphi)$ reads A (W/m^2) at ν_0, so the peak-to-peak swing is $2A$.

Derivation. With the mean removed, the windowed DFT of the record is [39]

$$X[k] = \sum_{n=0}^{N-1} w[n]\,(x[n] - \bar{x})\,e^{-i2\pi kn/N}, \qquad \nu_k = k\,\frac{f_s}{N}, \quad k = 0, \ldots, \lfloor N/2 \rfloor. \tag{6.17}$$

For a sinusoid $x[n] = A\cos(2\pi k_0 n/N + \varphi)$ on bin k_0,

$$X[k_0] = \frac{A}{2}e^{i\varphi}\sum_n w[n] + \frac{A}{2}e^{-i\varphi}\sum_n w[n]\,e^{-i4\pi k_0 n/N} \approx \frac{A}{2}e^{i\varphi}\sum_n w[n], \tag{6.18}$$

because the second sum is the window's transform $2k_0$ bins from its centre (exactly zero for the periodic cosine-sum windows when $2k_0$ exceeds their few non-zero coefficients). Hence the single-sided amplitude

$$\hat{A}_k = \frac{2\,|X[k]|}{\sum_n w[n]} = \frac{2\,|X[k]|}{N \cdot \mathrm{CG}} \quad (k \neq 0,\ k \neq N/2), \tag{6.19}$$

without the factor 2 at DC and at the Nyquist bin, whose cosine has no mirror image. For a rectangular window, Parseval's theorem then reads

$$\frac{1}{N}\sum_n (x[n] - \bar{x})^2 = \sum_{0<k<N/2} \frac{\hat{A}_k^2}{2} + \hat{A}_{N/2}^2, \tag{6.20}$$

the variance of the record is the sum of the powers of its sinusoids.

Algorithm.

1. put the record on the uniform grid (Section 6.5) and subtract its mean;
2. multiply by the window (`scipy.signal.get_window`, periodic form);
3. real FFT (`scipy.fft.rfft`, all cores; CuPy on a GPU with `--gpu`);
4. scale by $2/\sum w$, halve DC and (for even N) the Nyquist bin;
5. frequencies $\nu_k = k f_s/N$ (`scipy.fft.rfftfreq`).

Parameters and limits. The resolution is $\Delta\nu = f_s/N$, the inverse of the record length; zero padding interpolates the same spectrum but does not separate closer lines. A line between two bins reads low by the scalloping loss (−1.42 dB for Hann at half a bin); the chirp-Z zoom removes it. With `--correct-probe` the amplitude is divided by the probe response (Section 6.12). Verification: on-bin sinusoids read exactly for five windows (error $< 10^{-15}$), the amplitude equals $\sqrt{2}\times$`scipy.signal.periodogram` and a brute-force DFT, the Nyquist bin and Parseval's sum are exact.

6.8 Welch power spectral density

Purpose. How envelope power is distributed over frequency, with a smooth, statistically stable estimate: noise floors, broadband modulation, and weak lines that the single FFT shows only as a ragged noise floor.

Why averaging is needed. The periodogram of the whole record, $|X[k]|^2$ suitably scaled, is not a consistent estimator: for noise its standard deviation at each frequency equals its mean, however long the record is. Welch's method [11] trades resolution for stability: the record is cut into K overlapping segments, each segment's periodogram is computed, and the periodograms are averaged.

Definition. With segments $x_m[n] = x[n + mD]$ of L samples (`--nperseg`) and step $D = L/2$ (50 % overlap),

$$\hat{P}(\nu_k) = \frac{c_k}{K f_s \sum_n w[n]^2} \sum_{m=0}^{K-1} \left| \sum_{n=0}^{L-1} w[n](x_m[n] - \bar{x}_m) e^{-i2\pi kn/L} \right|^2, \qquad c_k = \begin{cases} 1 & k = 0,\ L/2 \\ 2 & \text{otherwise,} \end{cases} \tag{6.21}$$

in (unit)^2/Hz, e.g. (W/m^2)^2/Hz.

Normalisation. For white noise of variance σ^2, $\mathrm{E}|X_m[k]|^2 = \sigma^2 \sum_n w[n]^2$, so $\hat{P} = 2\sigma^2/f_s$ and $\int_0^{f_s/2} \hat{P}\, d\nu = \sigma^2$ (Parseval). More generally, for any data, $\sum_k \hat{P}_k f_s/L$ equals the mean windowed segment power divided by $\sum w^2$ exactly; the verification checks this identity to 10^{-16}.

Variance and resolution. Averaging K independent periodograms divides the variance by K. Overlapping segments are correlated; with correlation c between neighbours (Section 6.6) the normalised variance is approximately

$$\frac{\operatorname{var} \hat{P}}{P^2} \approx \frac{1}{K}\left(1 + 2c^2\right), \tag{6.22}$$

$1.056/K$ for Hann at 50 % overlap ($c = 0.167$), so overlap almost doubles the number of averages for the same record at little cost. The price is resolution: the effective bandwidth of each estimate is $\mathrm{ENBW} \cdot f_s/L$ ($1.50\, f_s/L$ for Hann), coarser than the single FFT's f_s/N.

Algorithm and parameters.

1. take the gap-free blocks of the uniform grid (no segment crosses a gap);
2. choose L: by default the largest power of two not above a quarter of the longest block, which with 50 % overlap gives at least seven segments;
3. for each block, segment starts every $D = L/2$ samples; process 512 segments at a time (bounded memory; the record may be a disk map);
4. remove each segment's mean, window, real FFT, accumulate $|X|^2$;
5. divide by $K f_s \sum w^2$ and double all bins except DC and Nyquist.

A spectral line of amplitude A appears as a peak whose *area*, not height, is its power $A^2/2$; read amplitudes from the FFT or chirp-Z pages, and use Welch for floors and broadband content. Verification: equal to `scipy.signal.welch` for four windows and odd and even L (error $< 2 \times 10^{-15}$), equal to the segment-weighted mean of SciPy's estimate per block for gappy data, and Parseval exact.

6.9 Chirp-Z zoom

Purpose. Read the frequency and amplitude of a line precisely, for example a rotation rate, by evaluating the spectrum on a fine grid in a narrow band, without the scalloping loss of the FFT grid.

The transform. The windowed record has the discrete-time Fourier transform (DTFT)

$$X(\nu) = \sum_{n=0}^{N-1} w[n](x[n] - \bar{x})\, e^{-i2\pi\nu n/f_s}, \tag{6.23}$$

a continuous function of ν; the FFT samples it only at kf_s/N. The chirp-Z transform [12] evaluates the z-transform

$$X_k = \sum_{n=0}^{N-1} x[n]\, z_k^{-n}, \qquad z_k = A\, W^{-k}, \quad k = 0, \ldots, M-1, \tag{6.24}$$

on an arc of the z plane. For a band from ν_1 to ν_2 on the unit circle, $A = e^{i2\pi\nu_1/f_s}$ and $W = e^{-i2\pi\Delta\nu/f_s}$ with $\Delta\nu = (\nu_2 - \nu_1)/(M-1)$, so $z_k = e^{i2\pi(\nu_1 + k\Delta\nu)/f_s}$ and $X_k = X(\nu_1 + k\Delta\nu)$: M samples of the DTFT, any M, anywhere.

Bluestein's algorithm. Direct evaluation costs NM operations. With the identity $nk = \frac{1}{2}[n^2 + k^2 - (k-n)^2]$ [19],

$$X_k = W^{k^2/2} \sum_{n=0}^{N-1} \left(x[n]\, A^{-n} W^{n^2/2}\right) W^{-(k-n)^2/2}, \tag{6.25}$$

a convolution of the "chirped" sequence $y[n] = x[n]A^{-n}W^{n^2/2}$ with the chirp $h[m] = W^{-m^2/2}$. The steps are:

1. form $y[n]$, $n = 0, \ldots, N-1$;
2. form $h[m]$ for $m = -(N-1), \ldots, M-1$;
3. convolve them with FFTs of length $L_{\mathrm{FFT}} \geq N + M - 1$;
4. multiply the result by $W^{k^2/2}$ for $k = 0, \ldots, M-1$.

The cost is $O((N+M)\log(N+M))$. The program calls `scipy.signal.zoom_fft(x, [`ν_1`,`ν_2`], m=M, fs=`f_s`, endpoint=True)` [31], which implements exactly this, and scales the result by $2/\sum w$ as in (6.19).

Resolution versus sampling density. The zoom samples the same windowed spectrum more densely; it does *not* improve the resolution, which stays f_s/N: two lines closer than about two bins (the Hann main lobe is four bins wide between nulls) remain one peak. What improves is the *reading* of an isolated line: its frequency to a small fraction of a bin and its amplitude without scalloping loss. Zero padding the FFT by a factor P samples the DTFT at the same density everywhere at a cost $O(PN \log PN)$; the zoom spends all M points in the band of interest.

Parameters. By default the band is ±16 FFT bins around the strongest line and $M = 4096$ (`--zoom-points`), a spacing of 1/128 bin; `--zoom f1:f2` sets the band. The refined values appear in the summary as `f_hz_czt` and `amp_czt`. Verification: equal to a brute-force DTFT on 257 zoom frequencies (error 8×10^{-15}), equal to the FFT on the bin frequencies, and a line 0.37 bin off the grid read to better than 2×10^{-3} bin and 3×10^{-3} in amplitude.

6.10 Lomb–Scargle periodogram

Purpose. The amplitude spectrum of a record whose time stamps are not evenly spaced (jitter, gaps, timer logs) without resampling it, so that interpolation neither smooths the data nor invents values in gaps.

Least-squares model. At each angular frequency $\omega = 2\pi\nu$ the centred samples y_j (mean removed) at times t_j are fitted by least squares [38, 13] with

$$y_j \approx a\cos\omega(t_j-\tau) + b\sin\omega(t_j-\tau). \tag{6.26}$$

The offset τ is chosen so that the two basis functions are orthogonal on the given stamps,

$$\tan 2\omega\tau = \frac{\sum_j \sin 2\omega t_j}{\sum_j \cos 2\omega t_j} \quad\Longrightarrow\quad \sum_j \cos\omega(t_j-\tau)\sin\omega(t_j-\tau) = 0. \tag{6.27}$$

Writing $C_j = \cos\omega(t_j-\tau)$ and $S_j = \sin\omega(t_j-\tau)$, the normal equations then decouple: $\hat{a} = \sum y_j C_j / \sum C_j^2$ and $\hat{b} = \sum y_j S_j / \sum S_j^2$, and the reduction of the residual sum of squares by the fit is

$$\Delta\chi^2 = \frac{\left(\sum_j y_j C_j\right)^2}{\sum_j C_j^2} + \frac{\left(\sum_j y_j S_j\right)^2}{\sum_j S_j^2} = 2P(\omega), \tag{6.28}$$

which defines the classical periodogram

$$P(\omega) = \frac{1}{2}\left\{\frac{\left[\sum_j y_j\cos\omega(t_j-\tau)\right]^2}{\sum_j \cos^2\omega(t_j-\tau)} + \frac{\left[\sum_j y_j\sin\omega(t_j-\tau)\right]^2}{\sum_j \sin^2\omega(t_j-\tau)}\right\}. \tag{6.29}$$

Amplitude. For $y_j = a\cos\omega(t_j-\tau) + b\sin\omega(t_j-\tau)$ with $a^2+b^2 = A^2$, orthogonality gives $\sum y_j C_j = a\sum C_j^2$ and $\sum y_j S_j = b\sum S_j^2$, so $P = \frac{1}{2}(a^2\sum C_j^2 + b^2\sum S_j^2)$. With samples spread over many cycles, $\sum C_j^2 \approx \sum S_j^2 \approx N/2$, hence

$$P \approx \frac{NA^2}{4} \quad\Longrightarrow\quad \hat{A} = \sqrt{\frac{4P}{N}}, \tag{6.30}$$

the amplitude the program reports, in the same unit as the FFT amplitude (6.19). On uniformly spaced stamps at the Fourier frequencies, $\sum C_j^2 = \sum S_j^2 = N/2$ exactly and $\hat{A}$ equals the rectangular-window FFT amplitude for any data; the verification confirms this to 5×10^{-14}.

Algorithm and parameters.

1. take the raw stamps and values (before any resampling) and subtract the mean;
2. frequency grid: from $1/T_{\text{span}}$ to $f_s/2$ (f_s from the mean interval) with $\min(4N, 20\,000)$ points, about four points per resolution width $1/T_{\text{span}}$;
3. for a chunk of frequencies (about 64 MiB of work arrays), compute τ, the four sums and P; on a GPU with `--gpu`;
4. $\hat{A} = \sqrt{4P/N}$.

The cost is $O(NM)$, much more than an FFT, so it runs automatically only when more than 0.1 % of the grid lies in gaps or the stamp jitter exceeds 1 % (`--lomb auto`); `on` and `off` force it. The program uses the classical form with a fixed mean; SciPy also offers a floating-mean (generalised) variant [14], which differs only for short records with uneven coverage. The significance of a peak (false-alarm probability) is not computed in this version. Verification: equal to `scipy.signal.lombscargle` (error 1.5×10^{-14}) and an irregularly sampled 0.8 amplitude read as 0.798 at the exact frequency.

6.11 Envelope spectrogram (short-time FFT)

For records of at least 16 Welch segments, the envelope is also shown as a short-time PSD (time × envelope frequency): the Welch segment PSD of (6.21) for each slice, without averaging.

It is computed with `scipy.signal.ShortTimeFFT(win, hop, fs, fft_mode="onesided2X", scale_to="psd")` [30]: windows centred at multiples of the hop, mean removed per slice, one-sided and PSD-scaled; only slices lying wholly inside the record are kept (`lower_border_end`, `upper_border_begin`). The hop is a quarter of the segment or larger, so that at most 2000 slices are produced. Time and frequency resolution trade against each other: a slice of L samples resolves f_s/L in frequency and L/f_s in time. It shows whether a modulation is steady or comes and goes. Verification: equal to `scipy.signal.spectrogram` on the 125 common slices (error 6×10^{-16}).

6.12 Probe correction

`--correct-probe` divides out the averaging (6.15) where that is possible. A plain inverse $1/|H|$ grows without bound near the null at $1/T$ and amplifies noise, so the gain is limited to $g_{\max}$ (`--max-boost-db`, default 12 dB, $g_{\max} = 3.981$):

$$G(\nu) = \begin{cases} \min(1/|H(\nu)|,\ g_{\max}), & \nu T < 1, \\ \text{undefined (NaN)}, & \nu T \geq 1. \end{cases} \tag{6.31}$$

The exact inverse is used wherever it is at most $g_{\max}$; beyond that the line is under-corrected but never over-amplified, and nothing at or beyond the first null can be recovered. G is continuous. (Version 0.0.2 used a Wiener-type gain $h/(h^2 + \varepsilon)$ below $|H| = 1/g_{\max}$, which jumped by 1.94 dB at the switch; the verification found it and 0.0.3 replaced it.) For the 5 Hz probes $|H| > 1/g_{\max}$ up to the Nyquist frequency (−7.55 dB at 2.5 Hz), so the exact inverse applies throughout; the clip matters only for probes whose first null lies below their Nyquist frequency (EFD-1891 fast: 434.8 Hz against 450 Hz). Verification: a simulated 265 ms average of a 1.5 Hz modulation is restored to 4×10^{-6}; the gain never exceeds 12 dB and changes by at most 0.002 dB between neighbouring frequencies.

6.13 Peaks and their exact frequency

Peaks are local maxima of the amplitude spectrum (the probe-corrected one when correction is on) that stand at least `--snr` times (default 6) above the median spectrum level, with a prominence of (snr − 1) times the median, within `--dyn-range-db` (default 50 dB, 20 log10) of the strongest peak; at most 12, DC and the first bin excluded. Each peak frequency is refined by a parabola through the logarithms of the three bins around it, $\alpha = \ln \hat{A}_{k-1}$, $\beta = \ln \hat{A}_k$, $\gamma = \ln \hat{A}_{k+1}$. Fitting $p(x) = ax^2 + bx + c$ through $(-1, \alpha)$, $(0, \beta)$, $(1, \gamma)$ gives $c = \beta$, $b = (\gamma - \alpha)/2$, $a = (\alpha - 2\beta + \gamma)/2$, and the vertex

$$\delta = -\frac{b}{2a} = \frac{1}{2}\,\frac{\alpha - \gamma}{\alpha - 2\beta + \gamma}, \qquad \hat{\nu} = \nu_k + \delta\,\Delta\nu. \tag{6.32}$$

The parabola is exact for a Gaussian window and a close approximation for Hann (verified: 0.016 bin error for a line 0.3 bin off centre). The chirp-Z zoom then gives the frequency and amplitude of the strongest line.

6.14 Harmonic families and rotation rate

A rotating antenna passes its beam over the probe once per turn, which gives a line at the rotation frequency ν_0 and harmonics $k\nu_0$ from the beam shape; a keyed or scanned source does the same at its repetition rate. The algorithm is:

1. take every peak above two bins as a candidate fundamental ν_0;
2. its members are the peaks within $\max(1.5\,\Delta\nu,\ 0.01\,\nu_0)$ of a whole multiple $k\nu_0$, $k \geq 1$;
3. keep the candidate whose members have the largest total amplitude, remove its members, and repeat (three families at most).

Each family is reported with ν_0, the period $1/\nu_0$ and the rate $60\,\nu_0$ in rpm. In Figure 8 the 12 rpm family (0.2 Hz and its harmonics) and the separate 0.7 Hz source are found this way.

6.15 Spectrograms: envelope spectrum per frequency bin and channels

For a spectrogram $L(t_m, f_j)$ (SignalShark, EHP-50 TS folder), the levels are converted to linear power $S(t_m, f_j)$ (Section 6.3), put on a uniform time grid at the mean spectrum interval, and transformed along time for every bin j at once, with the scaling of (6.19):

$$\hat{A}_j(\nu_k) = \frac{2}{\sum_m w[m]} \left| \sum_m w[m](S(t_m, f_j) - \bar{S}_j) e^{-i2\pi km/N_t} \right| \tag{6.33}$$

(DC and Nyquist rows not doubled; version 0.0.2 doubled the Nyquist row, which the verification found). The *modulation map* (Figure 12) shows this in dB relative to its maximum (20 log10), and the strongest envelope frequency of every bin is plotted separately.

Channels and channel power. *Channels* are analysed like probe logs, using the power summed over a band: the whole span (`total`) and the strongest spectral peaks (`--rf-channels auto:N`, ±2 bins or ±1.5 RBW) or bands in Hz (`--rf-channels f1:f2,f3:f4`). A trace bin holds the power that passes the resolution filter centred on it, so a carrier appears in every bin under the filter. For a Gaussian filter of 3 dB bandwidth B (power response $e^{-4\ln 2\,(f-f_c)^2/B^2}$), the bins of a carrier of power P_c, spaced Δf, sum to

$$\sum_j P_j \approx \frac{P_c}{\Delta f} \int_{-\infty}^{\infty} e^{-4\ln 2\, f^2/B^2}\, df = \frac{P_c\, B}{\Delta f} \sqrt{\frac{\pi}{4\ln 2}} = \frac{P_c\, B \cdot 1.0645}{\Delta f}, \tag{6.34}$$

so the absolute channel power is $P_c = \sum_j P_j\, \Delta f / (B \cdot \kappa)$ with the noise-bandwidth factor $\kappa = 1.0645$ for a Gaussian filter. With `--rbw-hz` (and `--enbw-factor` for another filter shape) the program applies this; without it the channel level is the plain bin sum, too high by $B\kappa/\Delta f$, and the summary says so. Envelope frequencies are not affected either way. Verification: a carrier of 10^{-3} W/m^2 on a 4× oversampled Gaussian-RBW trace is recovered to 10^{-4}. Images larger than 1500 pixels per side are reduced by taking the maximum of neighbouring cells for display; the full-resolution arrays go to the `.npz` file.

7 Command-line reference

Table 14 is generated from the program's own argument parser, so it always matches `python emf_envelope_fft_v0_0_4.py --help`.

Table 14: Command-line options (from the program's parser).

Option	Default	Description
`-h, --help`	–	show the help text and exit
`inputs ...`	–	files or EHP-50 TS folders (none: the GUI opens)
`--gui`	–	open the GUI with the given inputs listed
`--probe TEXT`	–	probe model (EFD-4091, EFD-0691, EF0691, EFD-9091, EFD-1891F); sets fs and T_int defaults
`--source TEXT`	`auto`	force a reader instead of auto-detection (choices: auto, fieldman, tsx, nbm, ehp, signalshark, xlsx)
`--fs FLOAT`	–	sample rate (Hz) when the file has no time column
`--t-int FLOAT`	–	probe integration time (s), overrides the catalogue
`--quantity TEXT`	`power`	quantity transformed: power (linear in sources), field or dB (choices: power, field, db)
`--window TEXT`	`hann`	SciPy window name (hann, blackmanharris, flattop, ...)
`--nperseg INT`	–	Welch segment length (samples)
`--zoom TEXT`	–	CZT zoom band 'f1:f2' in Hz (default: around the strongest peak)
`--zoom-points INT`	`4096`	number of CZT points in the zoom band

Table 14 (continued)

Option	Default	Description
`--lomb TEXT`	`auto`	Lomb-Scargle on the raw stamps (auto: with gaps or jitter) (choices: auto, on, off)
`--correct-probe`	–	undo the integration roll-off
`--max-boost-db FLOAT`	`12.0`	largest gain the correction may apply
`--rf-channels TEXT`	`auto:3`	spectrogram channels: 'auto:N' or 'f1:f2,f3:f4' (Hz)
`--antenna-factor-db FLOAT`	–	AF (dB/m) for dBm/dBuV spectrogram levels
`--rbw-hz FLOAT`	–	spectrogram resolution bandwidth (Hz): absolute channel power
`--enbw-factor FLOAT`	`1.064467019`	noise bandwidth / RBW of the analyzer filter (1.0645 = Gaussian)
`--display-unit TEXT`	`W/m2`	display units, one per family, comma separated, e.g. 'mW/cm2,dBuV/m,mA/m' (--list-units; data files stay in SI units)
`--time-display TEXT`	`power density`	what time plots and time pages show (E/H of power-density logs: plane wave, Z0) (choices: power density, E field, H field, logged field, analysed quantity)
`--rf-level-display TEXT`	`power density`	spectrogram levels of power-density inputs as power density or plane-wave E/H (choices: power density, E field, H field)
`--veusz-units TEXT`	–	extra units for the Veusz documents, e.g. 'W/m2,dBm/m2,dBuV/m'
`--veusz-layout TEXT`	`sections`	several units: one document with a section per unit, or one document per unit (choices: sections, files)
`--list-units`	–	print every display unit and exit
`--z0 FLOAT`	`376.730313668`	wave impedance (ohm); 377 matches Narda displays
`--snr FLOAT`	`6.0`	peak threshold over the median level (linear)
`--dyn-range-db FLOAT`	`50.0`	ignore peaks more than this many dB below the strongest
`--out TEXT`	–	output folder (default <input>_fft)
`--jobs INT`	`1`	parallel processes for several inputs
`--gpu`	–	use CuPy/CUDA FFTs when available
`--render`	–	PDF/PNG via vszh5_writer.py + Veusz
`--pickle`	–	also offload the full result to .pkl
`--time-col TEXT`	–	time column name/index ('Date+Time' joins two)
`--value-col TEXT`	–	value column name/index
`--unit TEXT`	–	unit of the values, e.g. 'V/m', 'mW/cm2', 'dBuV/m'
`--sep TEXT`	–	delimiter (default: sniffed)
`--decimal TEXT`	–	decimal mark (default: sniffed)
`--dayfirst`	–	dates are day-first
`--h5-level TEXT`	–	SignalShark .h5: level dataset path
`--h5-freq TEXT`	–	SignalShark .h5: frequency dataset path
`--h5-time TEXT`	–	SignalShark .h5: time dataset path
`--inspect`	–	only report how each input is read
`--live FILE`	–	follow a growing file and update live
`--window-s FLOAT`	`120.0`	live: rolling window (s)
`--update FLOAT`	`1.0`	live: update interval (s)
`--live-seconds FLOAT`	–	live: stop after this many seconds
`--no-gui`	–	live: console output only
`--selftest DIR`	–	run the built-in self-test (optionally keep outputs in DIR)
`--verify`	–	verify every transform and unit conversion (analytic, reference, identity checks)
`--version`	–	show program's version number and exit

8 Output files

Results go to the folder `<stem>_fft` beside the input, or to `--out`.

Table 15: Output files (folder <stem>_fft).

File	Content	Written
<stem>_envelope_fft.vszh5	Veusz document: all datasets and pages	always
<name>_amplitude.csv	f_hz, amplitude [, amplitude_corrected, amplitude_X/Y/Z]	per series / channel
<name>_welch.csv	f_hz, psd (unit^2/Hz)	per series / channel
<name>_zoom.csv	f_hz, amplitude (chirp-Z)	when a peak or --zoom
<name>_lomb.csv	f_hz, amplitude (Lomb-Scargle)	gaps/jitter or --lomb on
<stem>_modulation.npz	full-resolution spectrum, modulation image and dominant modulation per bin	spectrograms
<stem>_summary.json	peaks, families, warnings, settings	always
<stem>_result.pkl	the whole result object	--pickle
<stem>_envelope_fft_<unit>.vszh5	the document in one extra unit	--veusz-units with --veusz-layout files
<stem>_envelope_fft.pdf	all pages, bookmarked	--render
<stem>_envelope_fft_pNN_<page>.png	one image per page (150 dpi)	--render
<input>.live.f64	live mode: every sample, float64 (t, value) pairs	--live

8.1 The Veusz document

The .vszh5 file is Veusz's native HDF5 format, written directly by vszh5_writer.py (Appendix A.5): the datasets with their types and the plots as the Veusz command script, so the file opens with *File → Open* and every plot can be edited there. It is written to <name>.part and moved into place at the end, so an open copy is never half-written. Tables 16 and 17 list the pages; the page titles are also the PDF bookmark titles.

Table 16: Pages written for every time series (prefix ch<k>_ for spectrogram channels).

Page	Bookmark title	Content	When
summary	Summary (units, timing, peaks, warnings)	text: input, units, timing, probe model, peaks, families, notes	always
overview	Overview (time, spectrum, PSD, zoom)	four graphs: time series, amplitude spectrum, Welch PSD, chirp-Z zoom	always
time	Logged field vs time	the logged field (E, H or B) in its family's unit; a date axis when the start is known	always
density	Power density vs time	power density against time in the power-density unit	S known
efield	Electric field E vs time (plane wave)	E of the equivalent plane wave, sqrt(S Z0), in the E-field unit	E chosen, not logged
hfield	Magnetic field H vs time (plane wave)	H of the equivalent plane wave, sqrt(S/Z0), in the H-field unit	H chosen, not logged
amplitude	Envelope amplitude spectrum (FFT)	single-sided amplitude spectrum, log scale; probe-corrected and X/Y/Z traces	always
welch	Welch power spectral density	Welch PSD in dB, number and length of segments in the title	always
zoom	Chirp-Z zoom around the strongest peak	chirp-Z samples (line) and FFT bins (markers) around the strongest line	a peak

Table 16 (continued)

Page	Bookmark title	Content	When
`lomb`	Lomb-Scargle periodogram (raw stamps)	Lomb-Scargle amplitude from the raw stamps against the FFT	Lomb on
`probe`	Probe integration model |sinc|	|sinc(f T)| of the probe averaging in dB	T known
`spectrogram`	Envelope spectrogram	image: time x envelope frequency, PSD in dB (60 dB range)	long logs

Table 17: Overview pages of a spectrogram (SignalShark, EHP-50 TS), before the channel pages.

Page	Bookmark title	Content
`summary`	Summary (units, channels, warnings)	text: input, spectra, levels, channels with their rates, notes
`spectrum`	Mean and max-hold spectrum	mean and max-hold level per frequency (dB re W/m^2 or as given)
`spectrogram`	Spectrogram (level vs time and frequency)	image: frequency x time, level in dB
`modulation_map`	Envelope spectrum per frequency bin	image: frequency x envelope frequency, dB rel. maximum
`dominant_modulation`	Strongest envelope frequency per bin	strongest envelope frequency of every bin

8.2 Several units in the Veusz output

The pages are written in the display units (`--display-unit`, the GUI *Units and display* group). `--veusz-units` (GUI: *Veusz documents in several units*) adds further units; `--veusz-layout` chooses how:

`sections` (default) one document. After the pages in the display units comes one section per extra unit, holding its own summary page and only the pages whose values are in that unit's family (Table 18): an extra power-density unit gets the overview, power density against time and all spectra; an extra E or H unit gets the field against time (and, for spectrograms, the RF levels as that field); a unit of a family the record does not have gets no pages. Page names start with `u<k>__` and value datasets with `u<k>_`; the time and frequency axes are written once and shared. A unit equal to the display unit of its family adds only the pages the first set lacks.

`files` one complete document per extra unit, `<stem>_envelope_fft_<unit>.vszh5` (e.g. `_dBmm2`, `_mAm`), in which that unit replaces the display unit of its family and the time and RF-level modes follow it.

The text datasets `display_unit` (one title per section, "no pages" marked) and `unit_sections` (JSON of each section's choices) record the units in the document. The same options apply in the GUI, in batch runs and to the live snapshots. The bookmark tree of a document written while this manual was built (synthetic FieldMan log, `--display-unit mW/cm2,dBuV/m --veusz-units W/m2,dBm/m2,mA/m`) was read back from the document with `vszh5_writer.page_names`:

```
Display units: mW/cm², dBµV/m
  Summary (units, timing, peaks, warnings)  (page 1)
  Overview (time, spectrum, PSD, zoom)  (page 2)
  Logged field vs time  (page 3)
  Power density vs time  (page 4)
  Envelope amplitude spectrum (FFT)  (page 5)
  Welch power spectral density  (page 6)
  Chirp-Z zoom around the strongest peak  (page 7)
  Lomb-Scargle periodogram (raw stamps)  (page 8)
  Probe integration model |sinc|  (page 9)
In W/m² (Power density)
```

```
  Summary (units, timing, peaks, warnings)  (page 10)
  Overview (time, spectrum, PSD, zoom)  (page 11)
  Power density vs time  (page 12)
  Envelope amplitude spectrum (FFT)  (page 13)
  Welch power spectral density  (page 14)
  Chirp-Z zoom around the strongest peak  (page 15)
  Lomb-Scargle periodogram (raw stamps)  (page 16)
In dBm/m² (Power density)
  Summary (units, timing, peaks, warnings)  (page 17)
  Overview (time, spectrum, PSD, zoom)  (page 18)
  Power density vs time  (page 19)
  Envelope amplitude spectrum (FFT)  (page 20)
  Welch power spectral density  (page 21)
  Chirp-Z zoom around the strongest peak  (page 22)
  Lomb-Scargle periodogram (raw stamps)  (page 23)
In mA/m (Magnetic field H)
  Summary (units, timing, peaks, warnings)  (page 24)
  Magnetic field H vs time (plane wave)  (page 25)
```

Table 18: Pages of an extra unit section (layout sections), by the family of the extra unit.

Extra unit of	Pages of a probe log or channel	Spectrogram overview
the analysed quantity's family	summary, overview, amplitude, welch, zoom, lomb, spectrogram (+ density for power density)	spectrum, spectrogram (levels in that unit)
power density (record in W/m2)	summary, density	spectrum, spectrogram
E or H	summary, time (if logged) or efield / hfield (plane wave)	spectrum, spectrogram as plane-wave E or H (power-density inputs)
a family the record lacks	none (marked 'no pages')	none
--veusz-layout files	a complete document per unit	complete, with the modulation pages

8.3 Rendering to PDF and PNG

With `--render`, `vszh5_writer.render_pages` drives Veusz in listen mode and writes one multipage PDF and one PNG per page (`<stem>_envelope_fft_pNN_<page>.png`) for every document. The PDF gets a bookmark tree: the pages of a single log as a flat list; for a spectrogram an *Overview* entry, then one entry per channel (*Channel 0: …*) holding its pages; with unit sections, one top-level entry per unit holding that structure. *Veusz → Render* in the GUI reads the section titles back from the document, so a document written earlier gets the same tree.

8.4 Summary file

`<stem>_summary.json` holds the input and quantity (`unit_in`, `quantity`, `quantity_note`), timing (`fs_hz`, `nyquist_hz`, `samples`, `duration_s`, `resolution_hz`, `start`, `t_int_s`), the Welch settings, `peaks` (`f_hz`, `amp`, `snr_db`, and `f_hz_czt`, `amp_czt` for the zoomed line), `harmonic_families` (`f0_hz`, `period_s`, `rpm`, `harmonics`), the `warnings`, the compute back end and the program version. For a spectrogram it holds the frequency range and bin count, the spectrum rate and one such block per channel.

8.5 Examples

Figures 5–15 were rendered by this program from the synthetic files of its self-test (the FieldMan example with `--display-unit mW/cm2`; Figures 13–15 with `--display-unit mW/cm2,dBuV/m --veusz-units W/m2,dBm/m2,mA/m`). Every document starts with a summary page and gives each measurement an overview page (vszh5_writer `text_page` and `grid_page`, Appendix A.5).

fieldman_efd4091_tsx: summary (Display units: mW/cm², V/m)

```
Input            fieldman_efd4091_tsx   (reader fieldman, unit V/m)
Start            2026-10-15T14:03:05
Analysed         W/m² (S = E^2/Z0, Z0 = 376.730 ohm); spectra shown in mW/cm²
Time plot        Power density (mW/cm²)
Sampling         fs 5.00011 Hz, Nyquist 2.50005 Hz, 3000 samples, 599.987 s, resolution 0.001667 Hz
Welch            20 segments of 256 (resolution 0.01953 Hz)
Probe model      boxcar T = 265 ms, |H(Nyquist)| = -7.55 dB (20 log10), first null 3.774 Hz

Peaks            frequency (Hz)   amplitude (mW/cm²)   SNR (dB)   period (s)
                       0.199999          0.046568       41.1            5
                       0.399997          0.045042       40.8          2.5
                            0.6           0.04293       40.3       1.6667
                       0.700001          0.049103       41.5       1.4286
                       0.799998           0.03934       39.6         1.25
                              1          0.034847       38.5            1
                            1.2          0.030579       37.4      0.83333
                            1.4          0.026022       36.0      0.71428
                        1.60001           0.02168       34.4        0.625
                        1.80002          0.017301       32.5      0.55555
                        2.00002          0.012963       29.9          0.5
                        2.20002         0.0083941       26.2      0.45454

Harmonic families (rotation or repetition rate):
    f0 0.199999 Hz   period 5.00002 s   12 rpm   harmonics 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
    f0 0.700001 Hz   period 1.42857 s   42 rpm   harmonics 1

Notes:
    - 1.00 % of the grid lies in gaps (bridged linearly for the FFT; Welch uses gap-free blocks; see Lomb-
      Scargle)
    - time-stamp jitter 2.0 % of the interval
    - probe integration 265 ms: |H| = -7.6 dB at Nyquist 2.5 Hz; content between 2.5 and 3.774 Hz (first
      null) aliases into the band, attenuated only by |H|

Compute backend  CPU (NumPy/SciPy);  EMF Envelope FFT Analyzer 0.0.4
```

Figure 5: Veusz summary page: input, units, timing, probe model, peaks, harmonic families and notes, in the chosen display unit (here mW/cm^2). *(Synthetic data.)*

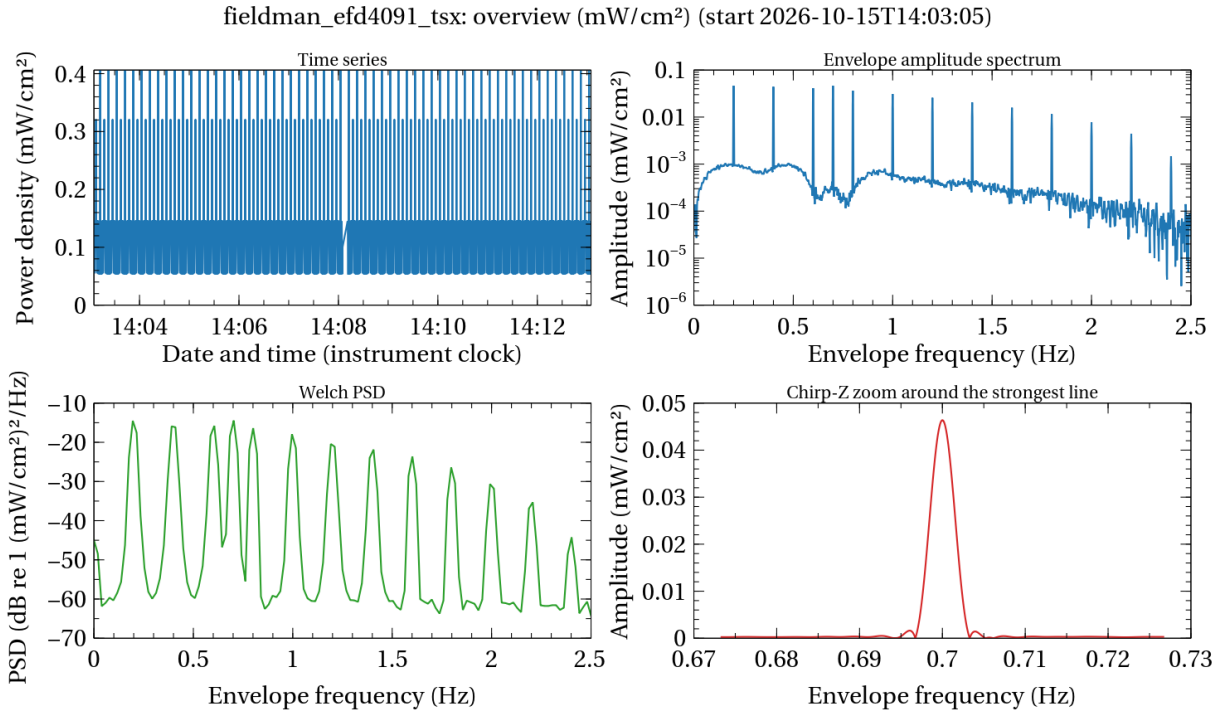


Figure 6: Veusz overview page: time series, amplitude spectrum, Welch PSD and chirp-Z zoom of one log. *(Synthetic data.)*

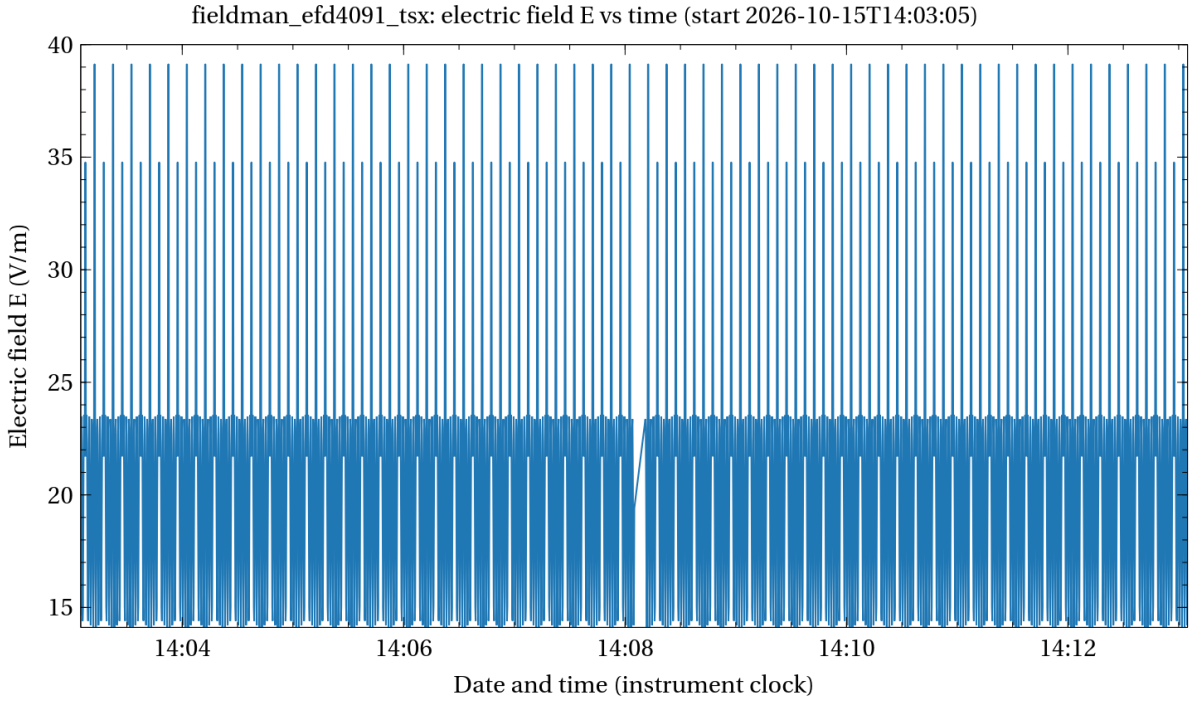


Figure 7: Synthetic FieldMan/Narda-TSX log (EFD-4091): field against the instrument clock on a date axis. The 6 s gap at 300 s is deliberate. *(Synthetic data.)*

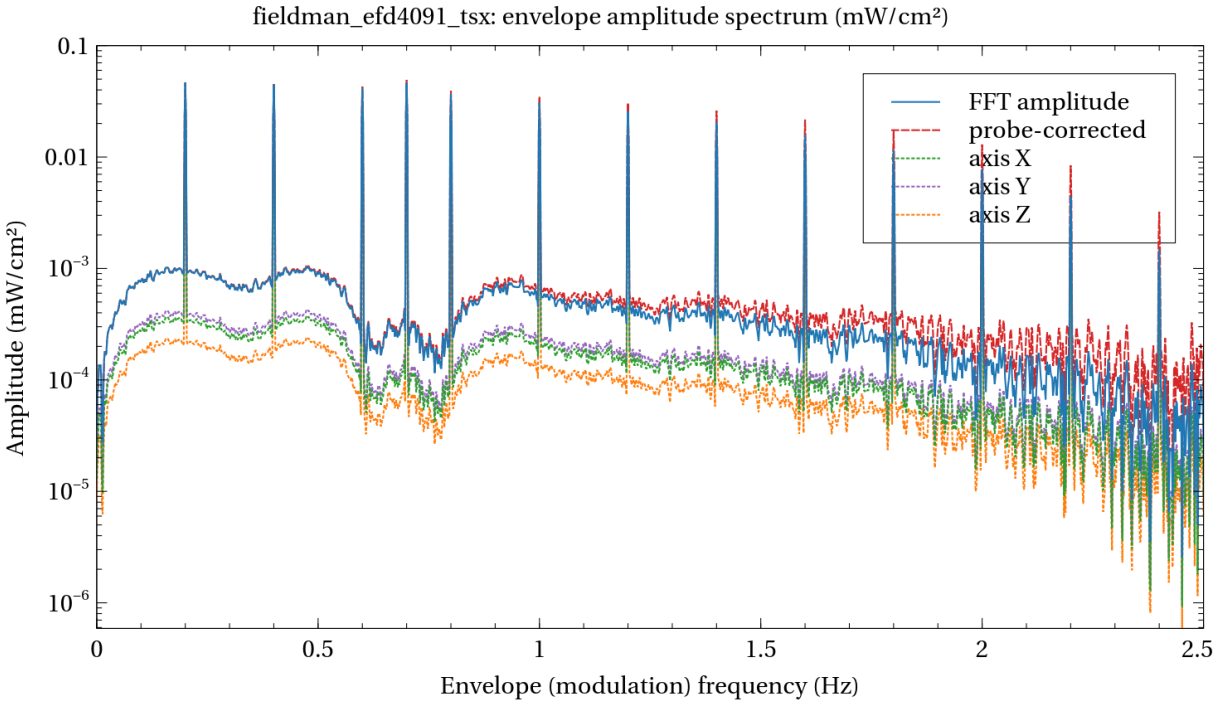


Figure 8: Envelope amplitude spectrum of the same log: the 12 rpm radar family (0.2 Hz and harmonics), the 0.7 Hz source, the probe-corrected trace (dashed) and the X/Y/Z axes (dotted). *(Synthetic data.)*

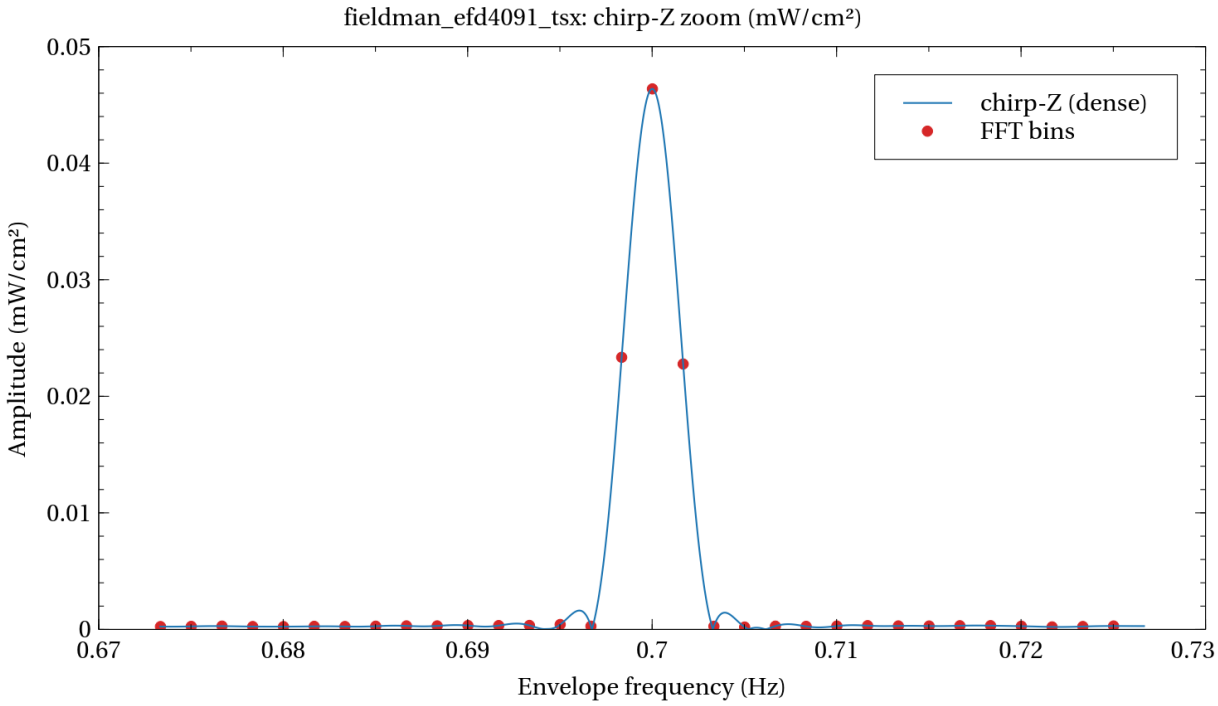


Figure 9: Chirp-Z zoom around the strongest line: the dense CZT samples (line) between the FFT bins (markers). *(Synthetic data.)*

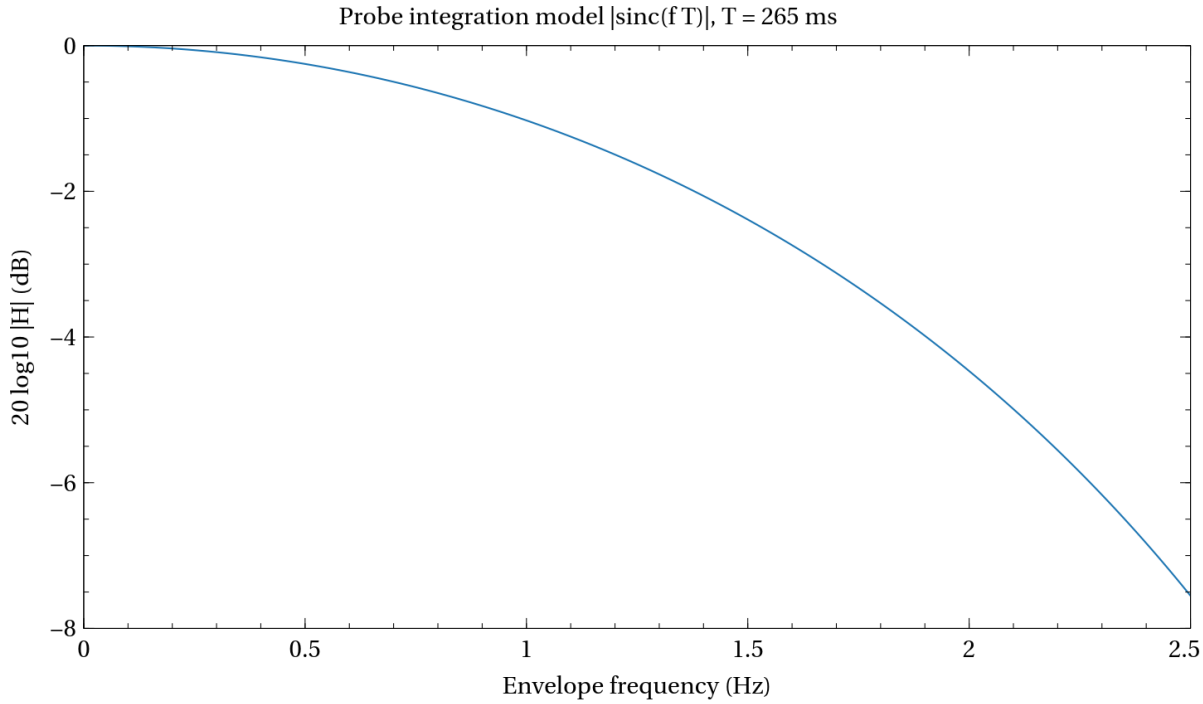


Figure 10: Probe integration model $|H(\nu)| = |\mathrm{sinc}(\nu T)|$ for $T = 265$ ms up to the Nyquist frequency. *(Synthetic data.)*

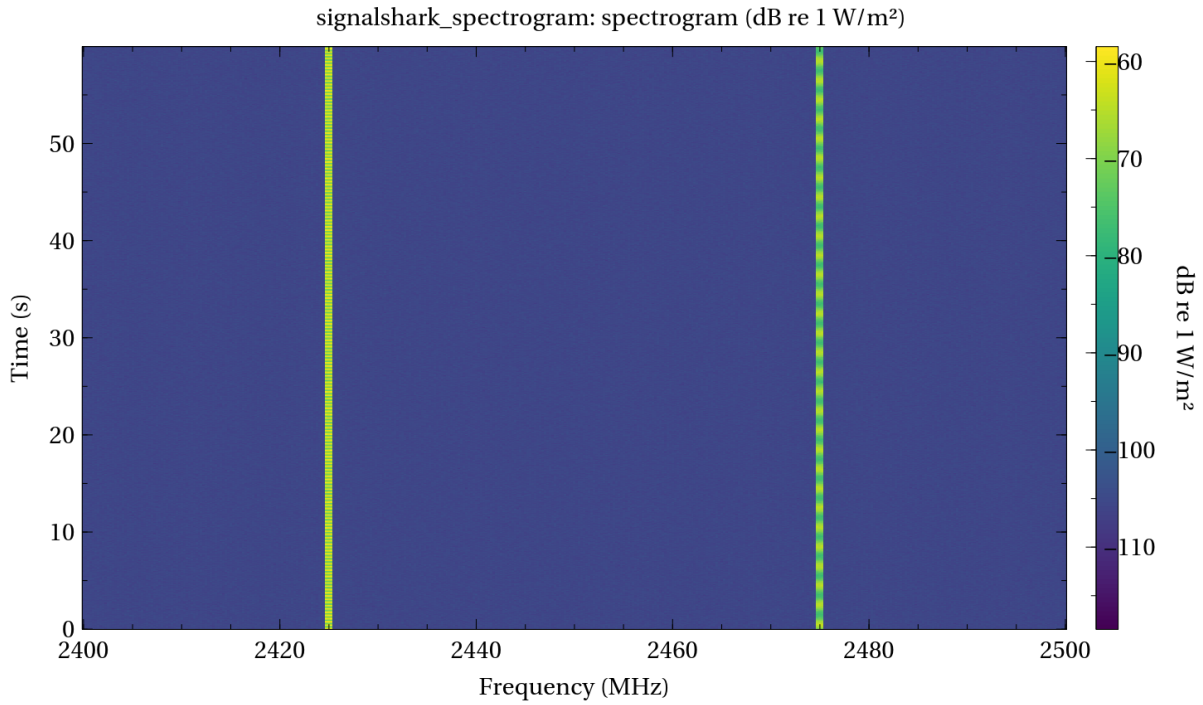


Figure 11: Synthetic SignalShark spectrogram: two carriers whose levels vary at 3 Hz and 0.5 Hz. *(Synthetic data.)*

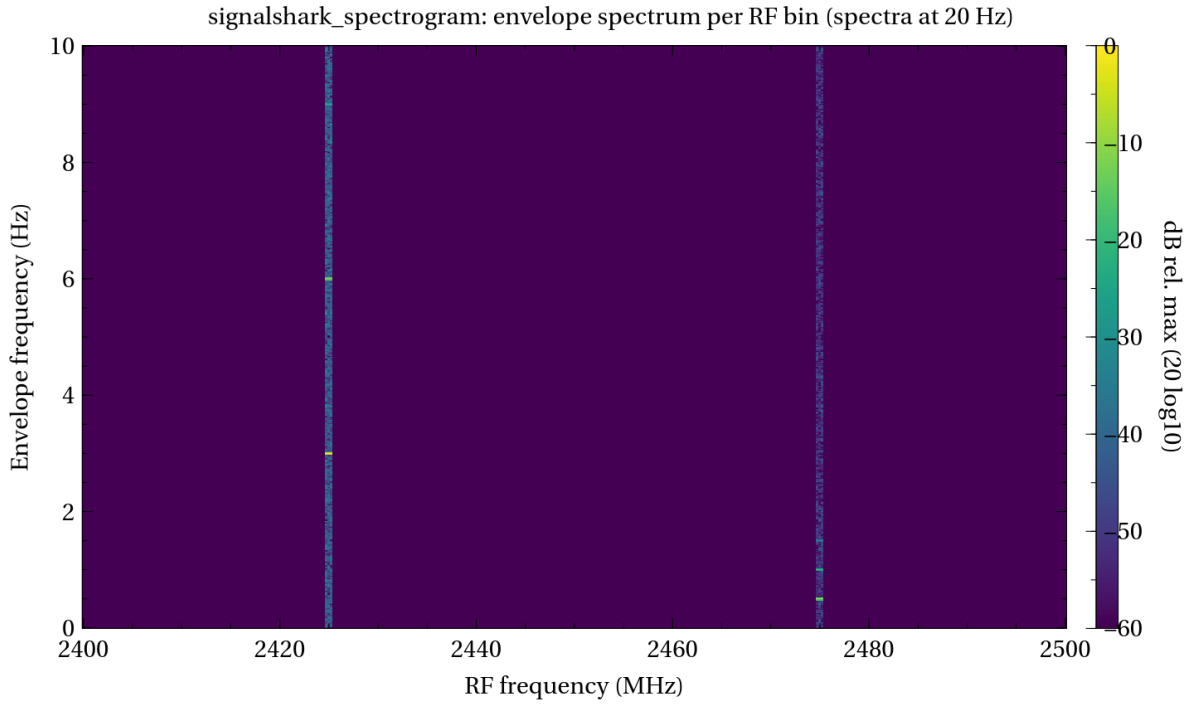


Figure 12: Envelope spectrum of every RF bin of that spectrogram: each carrier shows its own modulation frequency. *(Synthetic data.)*

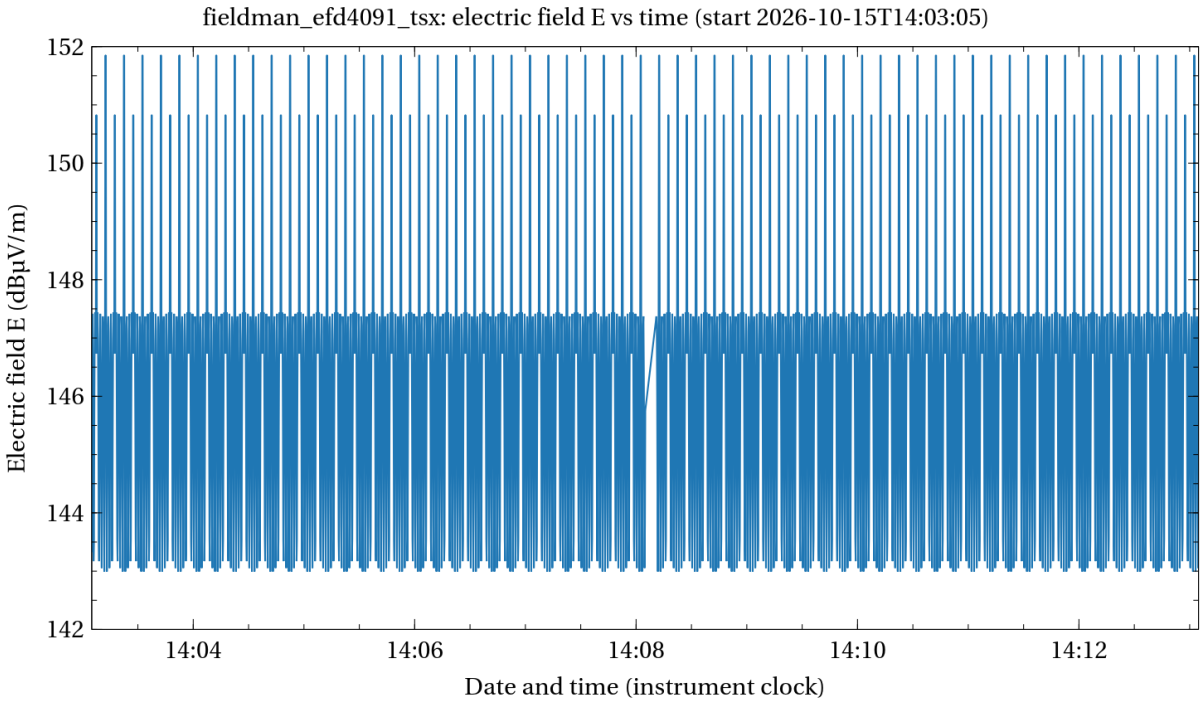


Figure 13: Logged field of the same log shown as E in dBµV/m (display units `mW/cm2,dBuV/m`). *(Synthetic data.)*

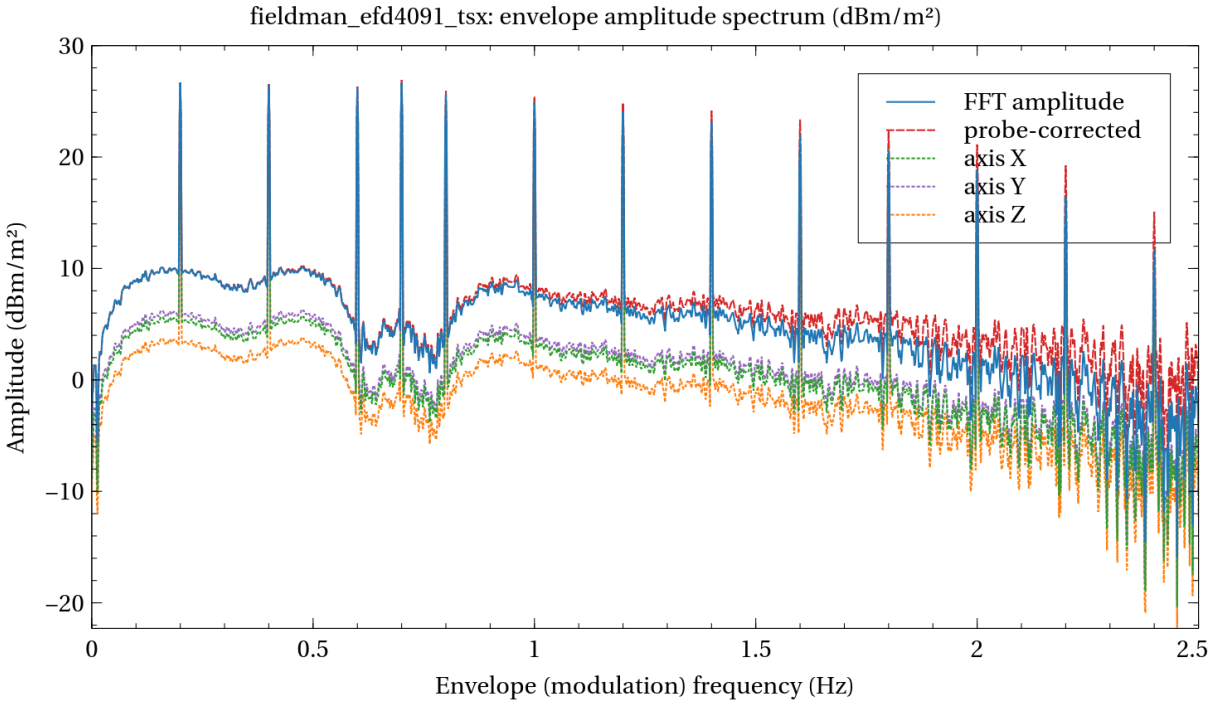


Figure 14: Page of an extra unit section (`--veusz-units dBm/m2`): the amplitude spectrum in dBm/m^2, $10\log_{10}$ of the amplitude in mW/m^2, on a linear axis. *(Synthetic data.)*

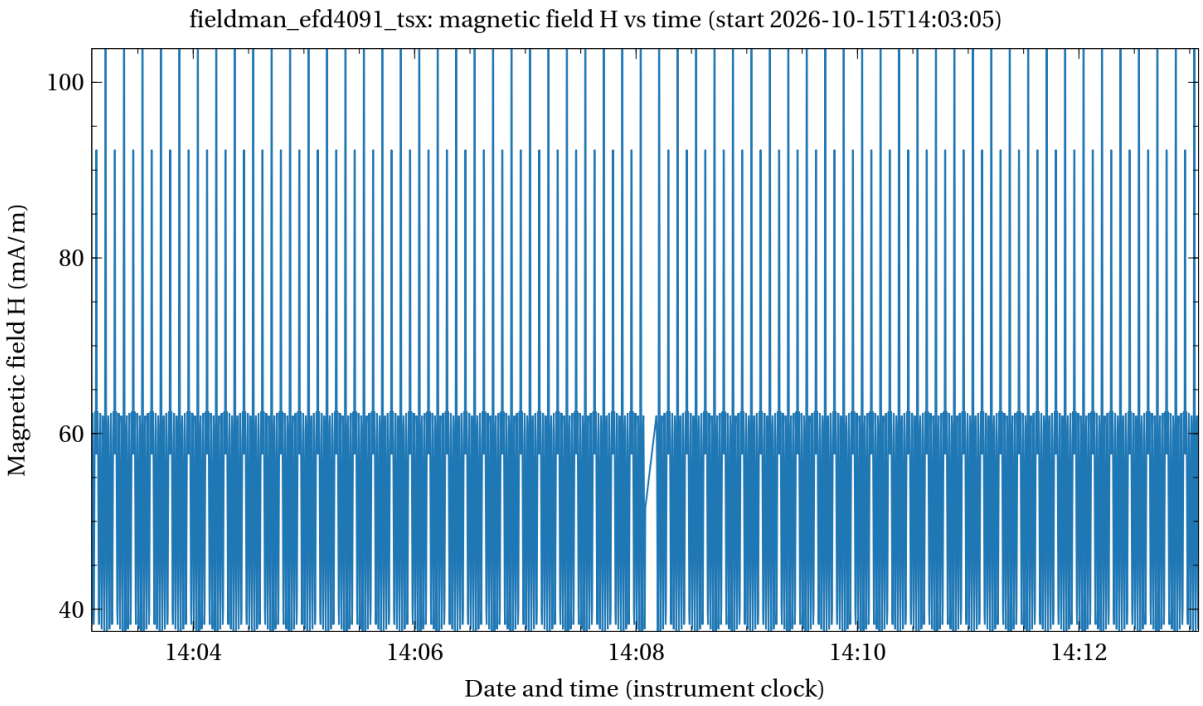


Figure 15: Page of the `mA/m` section: $H = \sqrt{S/Z_0}$ of the equivalent plane wave against time. *(Synthetic data.)*

9 Live mode

The *Live* tab of the GUI (or `--live FILE`) follows a text file that another program appends to. New complete lines are read every `--update` seconds (default 1 s; an unfinished last line waits for the next poll), kept in a ring buffer twice the size of the analysis window `--window-s` (default 120 s), and the envelope amplitude spectrum and Welch PSD of the latest window are recomputed and drawn. Every sample is also appended to `<file>.live.f64`, raw float64 pairs (time, value) that can be read back with `np.fromfile(path).reshape(-1, 2)`. The plots follow the unit control of the settings panel exactly as the Analysis tab does: the time plot in the *Time plot shows* mode, the spectra in the unit of the analysed quantity's family, the PSD in dB re 1 (unit)^2/Hz, re-plotted at once when a unit changes; the console mode (`--no-gui`) prints the peak amplitudes in that unit. *Save snapshot* analyses the current window fully and writes the usual output files, including the extra Veusz units and layout, and *Clear buffer* empties the RAM buffer (the disk log keeps every sample). `--no-gui` prints the strongest lines and families instead; `--live-seconds` stops after a time.

Neither FieldMan nor Narda-TSX documents a continuously written file or a streaming interface for the broadband probes [5, 9], so whether live mode works with them depends on how they write data; it was tested with a file written in real time by a script. NBM-550 remote output (`MEAS_START`, 5, 50 or 60 samples per second [6]) logged to a file by another tool is a documented route.



Figure 16: Live tab (dark theme) following a growing file with 2 Hz and 7 Hz envelope tones. *(Synthetic data.)*

10 Memory, parallel processing and GPU

- **Spill to disk**: a column longer than 20 000 000 samples is moved from RAM to a temporary raw file and used as a memory map; the files are deleted when the program exits. A 2.4-million-row, 900 Hz test log was analysed this way.
- **Chunks and batches**: text files are read 500 000 rows at a time; Welch processes 512 segments at a time; Lomb–Scargle about 64 MiB of frequencies at a time; the per-bin modulation spectra in batches of about 256 MiB.

- **Freeing memory**: `flush_memory()` runs the garbage collector and empties the CuPy memory pools; batch mode calls it after each file and after every GUI analysis run, and so do the *Clear buffer* button of the Live tab (empties the ring buffer; the disk log is kept) and *Edit → Free memory* (forgets the results shown and empties the live buffer). `--pickle` also writes the whole result object to `<stem>_result.pkl` (load it with `load_pickle`, only from files you trust).
- **Parallel**: `--jobs N` analyses N files in separate processes; each process then uses one FFT thread so that the cores are not oversubscribed. With one job, SciPy's FFTs use every core.
- **GPU**: `--gpu` moves the FFTs, Welch segments, modulation spectra and Lomb–Scargle sums to an NVIDIA GPU through CuPy [27]. Without CuPy or a CUDA device the program continues on the CPU and says why in the summary (`backend`).

11 Working in Spyder

The script is organised in Spyder cells, each opened by a banner `# %% ====` and a title line. Table 19 lists them with the functions and classes each defines (generated from the script). Run the cells from the top; the last cell is a commented scratch cell:

```
opts = Options(probe="EFD-4091", correct_probe=True, lomb="on",
               display_unit="mW/cm2,dBuV/m", veusz_units="dBm/m2,mA/m")
print(inspect_input(r"C:\path\to\export.csv", opts))
series = load_any(r"C:\path\to\export.csv", opts)
result = analyse_timeseries(series, opts)
print(result.peaks[:5], result.families)
print("\n".join(summary_lines(result, "uW/cm2,dBuV/m")))
write_outputs(result, Path(r"C:\path\to\out"), opts)
opts.veusz_units, opts.veusz_layout = "W/m2,uW/cm2", "files"     # other units,
write_veusz(result, Path(r"C:\path\to\out"), opts)             # no re-analysis
print_unit_table()
flush_memory()
```

Table 19: Spyder cells of emf_envelope_fft_v0_0_4.py.

No.	Cell	Line	Definitions
1	MODULE HEADER	6	*(constants, imports or comments)*
2	IMPORTS	329	*(constants, imports or comments)*
3	CONSTANTS, PROBE CATALOGUE AND UNITS	392	ProbeSpec, lookup_probe, parse_unit, to_power_like, to_field_like, DisplayUnit, UnitFamily, lookup_display_unit, _split_units, … (32 in all)
4	DATA MODEL	1036	TimeSeries, SpectrumSet, Options
5	MEMORY HELPERS (bounded RAM for long logs)	1140	_remove_spill_files, SpillBuffer, flush_memory, save_pickle, load_pickle
6	COMPUTE BACKEND (CPU / optional GPU)	1253	Backend
7	TEXT EXPORT SNIFFING AND READING	1294	TextLayout, _read_head, _is_number, _split, sniff_text, _pick_columns, TimeOrigin, _to_seconds, _origin_text, … (23 in all)
8	PRE-PROCESSING: TIMING, GAPS, UNIFORM GRID	1967	UniformSeries, make_uniform, analysis_quantity
9	SPECTRAL ESTIMATORS	2049	amplitude_spectrum, welch_blocks, czt_zoom, lomb_scargle, envelope_spectrogram, modulation_spectra, probe_response, correction_gain, correct_probe
10	PEAKS AND HARMONIC FAMILIES	2229	find_spectral_peaks, harmonic_families
11	ANALYSIS DRIVERS	2296	SeriesResult, _default_nperseg, analyse_timeseries, SpectrogramResult, _decimate_max, analyse_spectrumset
12	VEUSZ .vszh5 OUTPUT (through the author's vszh5_writer.py)	2632	_freq_unit, _outline_for, bookmark_outline, UnitView, primary_view, _focus_view, veusz_views, safe_unit_tag, _result_units, … (23 in all)

Table 19 (continued)

No.	Cell	Line	Definitions
13	OTHER OUTPUTS: CSV, NPZ, JSON SUMMARY, OPTIONAL RENDER	3292	_summary_series, _summary_spectrogram, _jsonable, _write_csv, write_outputs, write_veusz, render_documents, render_vszh5
14	BATCH (PARALLEL) PROCESSING	3451	analyse_path, output_folder, analyse_and_write, process_file, process_files
15	LIVE MODE: FOLLOW A GROWING FILE	3519	LiveTail, RingBuffer, live_update, live_record, _live_ring, run_live_console
16	GUI: QT BINDING (Qt.py or QtPy abstraction layer, PySide6 binding)	3711	_QtMissingMeta, _QtMissing, _import_qt
17	GUI: THEME MANAGER (light / dark / system, as in the QuickSyn controller)	3804	ThemeManager
18	GUI: SMALL WIDGETS AND HELPERS	3890	make_double_spin, make_int_spin, make_combo, import_qt_module, export_widget_svg, monospace_font, _finite_positive
19	GUI: PLOT PANELS (pyqtgraph)	3978	PlotPanel, ImagePanel
20	GUI: BACKGROUND TASKS (analysis, inspect, verification in a QThread)	4231	TaskThread, analysis_job
21	GUI: SETTINGS PANEL (inputs, analysis options, output, display units)	4300	SettingsPanel
22	GUI: RESULTS VIEW (plots, peaks, summary, inspect, verification)	4750	ResultsView
23	GUI: LIVE PANEL (follow a growing file)	5070	LivePanel
24	GUI: ABOUT DIALOG CONTENT	5254	_module_version, build_about_content
25	GUI: MAIN WINDOW	5323	MainWindow
26	GUI: ENTRY POINT	6115	launch_gui, run_live_gui
27	VERIFICATION OF TRANSFORMS AND UNIT CONVERSIONS (0.0.3)	6177	Check, verify_transforms
28	SELF-TEST WITH SYNTHETIC EXPORTS	6543	_synthetic_radar, _probe_filter, make_synthetic_files, self_test
29	COMMAND LINE	6751	build_parser, options_from_args, _unit_list_argument, print_unit_table, main
30	INTERACTIVE SCRATCH CELL (Spyder)	6906	*(constants, imports or comments)*

12 Verification

`--selftest` checks the estimators against known answers and then runs every reader on synthetic files written in the styles expected from the instruments (semicolon and decimal comma with German dates, a 1 s NBM timer log, an EHP-50 TS folder, a SignalShark-like HDF5 file and a converter CSV). Table 20 is the result of the run made while building this manual.

Table 20: Self-test result (program 0.0.4, run during this build).

Check	Measured	Result
streamed Welch == scipy.signal.welch	max rel err 3.6e-15	PASS
FFT amplitude of 0.8 sinusoid	0.802	PASS
CZT peak frequency	0.21000 Hz	PASS
Lomb-Scargle amplitude and frequency	0.799 at 0.2100 Hz	PASS
|sinc| at 1.5 Hz, T=265 ms	-2.39 dB	PASS
transform and unit verification (--verify)	54/54 checks	PASS
FieldMan/TSX	radar 12 rpm family: [12.0, 42.0]	PASS

Table 20 (continued)

Check	Measured	Result
FieldMan/TSX	0.7 Hz source found	PASS
NBM timer log	6 rpm: [6.0]	PASS
EHP-50 TS spectra folder	0.05 Hz modulation of 50 Hz line: [0.05, 0.05]	PASS
signalshark	per-channel envelope 0.5 and 3 Hz: [0.5, 3.0]	PASS
signalshark_csv	per-channel envelope 0.5 and 3 Hz: [0.5, 3.0]	PASS
.vszh5 layout (Veusz format 1)	signalshark_converted_envelope_fft.vszh5	PASS
Veusz sections	W/m2, dBm/m2 and mA/m sections in one document: 42 datasets; sections ['Display units: mW/cm², dBµV/m', 'In W/m² (Power density)', 'In dBm/m² (Power density)', 'In mA/m (Magnetic field H)']	PASS
Veusz values in mW/cm2 (x0.1) and dBm/m2 (10 log10(x1e3)) at the strongest line	0.04637 mW/cm2, 26.6624 dBm/m2	PASS
bookmarks grouped per unit section	['Display units: mW/cm², dBµV/m', 'In W/m² (Power density)']	PASS
Veusz one document per unit	fieldman_efd4091_tsx_envelope_fft.vszh5, fieldman_efd4091_tsx_envelope_fft_Wm2.vszh5, fieldman_efd4091_tsx_envelope_fft_dBmm2.vszh5, fieldman_efd4091_tsx_envelope_fft_mAm.vszh5	PASS

12.1 Verification of the transforms and unit conversions

`--verify` (also *Analysis → Verify transforms and units* in the GUI, and part of the self-test) checks every time-to-frequency transform and every unit conversion three independent ways:

analytic a signal whose answer is known exactly: on-bin sinusoids, a line half a bin off, hand-worked unit values, the sinc of a moving average, a Gaussian-RBW carrier;

reference an independent implementation: `scipy.signal.periodogram`, `welch`, `spectrogram` and `lombscargle` [10], or a brute-force DFT or DTFT sum;

identity a law the result must obey: Parseval's theorem, Lomb–Scargle equal to the FFT on a uniform grid, the zoom equal to the FFT on the bin frequencies, conversion round trips.

Table 21 is the result of the run made while building this manual (54 checks). It found, and version 0.0.3 fixed: the Nyquist row of the per-bin modulation spectra was doubled; the unit headers mW/m^2, µW/m^2 and dB(µV/m) were not recognised and dBm/m^2 was read as dBm; the dBm to dBµV constant was rounded to 107 dB; the probe-correction gain jumped by 1.94 dB. Version 0.0.4 added the group *Display units*: spellings, hand-worked values in every family, round trips of all 34 units, the 10/20 dB level rule, squared units, PSD offsets, the plane-wave relations $E/H = Z_0$ and $EH = S$, the conversion of 1 mW/cm^2 to E and H, RF levels in dBm/m^2, dBµV/m and dBW, and the per-family unit selection.

Table 21: Verification of transforms and units (program 0.0.4, run during this build).

Group	Method	Check	Measured	Limit	Result
Units	analytic	parse_unit on 25 header spellings	all recognised	all	PASS
Units	analytic	to_power_like: 14 hand-worked values (V/m, dBuV/m, W/m2, mW/cm2, A/m, ...)	0.00e+00	<= 1e-12	PASS
Units	analytic	dBm -> dBuV (+106.99 dB, 50 ohm) -> + AF -> E^2/Z0	0.00e+00	<= 1e-12	PASS
Units	analytic	dBm without AF -> mW (0 dBm = 1 mW); dBuV -> uV^2 (20 dBuV = 100 uV^2)	0.00e+00	<= 1e-12	PASS
Units	identity	round trip E -> S = E^2/Z0 -> E = sqrt(S Z0)	9.19e-17	<= 1e-12	PASS
Units	analytic	E in dBuV/m: 1 V/m = 120, 1 mV/m = 60	0.00e+00	<= 1e-09	PASS
Units	analytic	display units: 1 W/m2 = 1e3 mW/m2 = 1e6 uW/m2 = 0.1 mW/cm2 = 100 uW/cm2	0.00e+00	<= 1e-15	PASS
Units	identity	display units of a PSD scale with the factor squared	0.00e+00	<= 1e-15	PASS
Units	analytic	to_field_like: 1 mW/cm2 -> sqrt(10 Z0) = 61.38 V/m	0.00e+00	<= 1e-12	PASS
Display units	analytic	lookup_display_unit on 13 spellings (µ/µ/u, ^2/², dB(...))	all recognised	all	PASS

Table 21 (continued)

Group	Method	Check	Measured	Limit	Result
Display units	analytic	25 hand-worked values: 1 W/m2 = 0.1 mW/cm2 = 30 dBm/m2, 1 V/m = 120 dBuV/m, 1 mW = -30 dBW, ...	0.00e+00	<= 1e-12	PASS
Display units	identity	round trip to_display -> from_display for all 34 units	1.84e-15	<= 1e-12	PASS
Display units	analytic	level rule: x10 in value = +10 dB (power units), +20 dB (root-power units)	0.00e+00	<= 1e-12	PASS
Display units	identity	squared quantities: (k x)^2 for linear units; dB re 1 (ref)^2 of x^2 = level of x	2.84e-14	<= 1e-12	PASS
Display units	analytic	PSD in dB re 1 (unit)^2/Hz: mW/cm2 -20 dB, dBm/m2 (mW/m2) +60 dB, dBuV/m +120 dB	0.00e+00	<= 1e-12	PASS
Display units	identity	plane wave: E/H = Z0 and E x H = S for the time-plot conversions	3.35e-16	<= 1e-12	PASS
Display units	analytic	1 mW/cm2 as E = 155.760 dBuV/m and H = 162.924 mA/m	0.00e+00	<= 1e-12	PASS
Display units	analytic	RF levels: 0 dB(W/m2) = 30 dBm/m2 = 145.76 dBuV/m (plane wave); 0 dBm = -30 dBW	0.00e+00	<= 1e-09	PASS
Display units	analytic	unit_selection: each unit sets its own family (0.0.3 single keys accepted)	consistent	all	PASS
FFT	analytic	on-bin sinusoid amplitude, hann window	1.39e-16	<= 1e-09	PASS
FFT	analytic	on-bin sinusoid amplitude, hamming window	1.39e-16	<= 1e-09	PASS
FFT	analytic	on-bin sinusoid amplitude, blackmanharris window	1.39e-16	<= 1e-09	PASS
FFT	analytic	on-bin sinusoid amplitude, flattop window	2.78e-16	<= 1e-09	PASS
FFT	analytic	on-bin sinusoid amplitude, boxcar window	0.00e+00	<= 1e-09	PASS
FFT	reference	amplitude = sqrt(2 x scipy periodogram 'spectrum'), hann, random data	1.23e-15	<= 1e-10	PASS
FFT	reference	amplitude = sqrt(2 x scipy periodogram 'spectrum'), flattop, random data	7.94e-16	<= 1e-10	PASS
FFT	reference	brute-force DFT sum at 4 bins (Hann)	6.65e-14	<= 1e-10	PASS
FFT	analytic	Nyquist bin not doubled: cos(pi n) of amplitude 0.5 reads 0.5	0.00e+00	<= 1e-12	PASS
FFT	identity	Parseval (boxcar): sum A_k^2/2 + A_Nyq^2 = variance	1.11e-16	<= 1e-12	PASS
FFT	analytic	Hann scalloping at half a bin equals the window response (-1.42 dB)	-1.424 dB	<= 1e-02	PASS
Welch	reference	= scipy.signal.welch, hann, nperseg 256	1.04e-15	<= 1e-10	PASS
Welch	reference	= scipy.signal.welch, hamming, nperseg 255	9.32e-16	<= 1e-10	PASS
Welch	reference	= scipy.signal.welch, blackmanharris, nperseg 128	2.28e-15	<= 1e-10	PASS
Welch	reference	= scipy.signal.welch, flattop, nperseg 64	1.44e-15	<= 1e-10	PASS
Welch	reference	gap-aware: = segment-weighted mean of scipy welch per block	9.33e-16	<= 1e-10	PASS
Welch	identity	Parseval: sum P df = mean windowed segment power / sum w^2	1.11e-16	<= 1e-10	PASS
CZT	reference	= brute-force DTFT on the 257 zoom frequencies	8.33e-15	<= 1e-09	PASS
CZT	identity	zoom on the FFT bin frequencies = FFT amplitudes	8.10e-15	<= 1e-09	PASS
CZT	analytic	off-bin line (k0 + 0.37): frequency error / bin width	0.00e+00	<= 2e-03	PASS
CZT	analytic	off-bin line: amplitude error (no scalloping loss)	8.11e-04	<= 3e-03	PASS
Lomb-Scargle	reference	= scipy.signal.lombscargle, amplitude sqrt(4P/N)	9.24e-15	<= 1e-08	PASS
Lomb-Scargle	analytic	irregular stamps: amplitude 0.8 at 0.21 Hz	0.7988 at 0.21000 Hz	<= 2e-02	PASS
Lomb-Scargle	identity	uniform stamps at Fourier frequencies = boxcar FFT amplitude	3.99e-14	<= 1e-09	PASS
Spectrogram	reference	= scipy.signal.spectrogram on 125 common slices	7.64e-16	<= 1e-09	PASS
Probe model	reference	|sinc(f T)| = moving average of T at 1 MHz (closed form)	4.32e-12	<= 1e-06	PASS
Probe model	analytic	simulated 265 ms average of a 1.5 Hz modulation: ratio = |sinc|	0.75962 vs 0.75962	<= 1e-03	PASS
Probe model	analytic	correction restores the 0.5 modulation amplitude	3.70e-06	<= 1e-03	PASS
Probe model	identity	correction gain <= g_max, continuous, NaN from the first null	max 12.00 dB, largest step 0.0023 dB	<= 12 dB, step < 0.01 dB	PASS
Peaks	analytic	parabolic interpolation of an off-bin line (k + 0.3), Hann	0.0160 bin	<= 5e-02	PASS
Peaks	analytic	harmonic families: 0.2 Hz (12 rpm, 5 members) and 0.7 Hz	0.00e+00	<= 1e-09	PASS
Modulation map	identity	every bin = amplitude_spectrum of its power series (incl. Nyquist row)	1.47e-14	<= 1e-12	PASS
Units	analytic	channel power from an oversampled Gaussian-RBW trace = carrier (W/m2)	9.998960e-04 W/m2	<= 1e-03	PASS

Table 21 (continued)

Group	Method	Check	Measured	Limit	Result
Grid	analytic	900 Hz stamps rounded to 1 us: rate from the mean interval, samples untouched	1.11e-08	<= 1e-06	PASS
Grid	analytic	a 6 s gap gives two gap-free blocks and ~1 % bridged	2 blocks, 1.03 %	<= 2e-03	PASS

12.2 Further checks

Further checks (made for version 0.0.4 unless another version is named):

- Unit control in the GUI, offscreen, with Qt.py and QtPy: *View → Units* with six families, the amplitude axis in dBm/m^2 (linear axis, peak table equal to $10\log_{10}(10^3 A)$), all five time-plot modes, RF levels as dBµV/m, the settings round trip, *Write .vszh5 again* in sections and files layouts, and the live tab in dBµV/m and µW/cm^2 with a snapshot in two units.
- Every synthetic input in unusual families (EHP-50 TS field squared in (mV/m)^2 with pT, µT and dBµV/m sections; SignalShark levels as E; a field analysis in dBµV/m with H against time; an NBM log in µW/cm^2 with one file per unit): written without error, labels and section titles as expected. Rendered pages checked by eye: 1 mW/cm^2-scale peaks read 26.7 dBm/m^2 and the logged field 143–152 dBµV/m, as computed by hand.
- (0.0.3) Every CSV of the self-test compared with version 0.0.2: identical; the summaries differ only by the new channel-power note and by the last bit of floating-point values (about 10^{-16}).
- (0.0.3) Rendering in Veusz 4.2.1 of a probe log (9 pages, plots in mW/cm^2) and a SignalShark spectrogram (23 pages): summary text page, overview grid, date axis, line styles, markers, colour bars and the bookmark tree checked by eye and with pypdf.
- GUI, offscreen, with both Qt.py and QtPy over PySide6 (0.0.3, repeated for 0.0.4): analysis of three inputs in the worker thread, display units (axis labels and values in mW/cm^2), time-plot modes, light and dark themes, About content (author, version, licence), verification from the menu (all pass), plot export to PNG, SVG and CSV, peaks table, cursor readout, shortcut and probe dialogs, live tab following a file written at 50 samples per second with 2 Hz and 7 Hz tones (both found).
- A 2.4-million-row 900 Hz log through the spill-to-disk path (0.0.2): lines at 37.0 and 120.0 Hz found exactly.
- (0.0.3) Three files with `--jobs 3 --gpu` on a machine without CuPy: all analysed, CPU fallback reported.

13 Limitations and known issues

1. The readers are heuristic until real exports of FieldMan, Narda-TSX, NBM-TS, EHP-50 TS and SignalShark have been examined; use `--inspect`.
2. “EFD-4091E” is treated as the EFD-4091; no datasheet for an E variant was found.
3. The probe averaging is modelled as a boxcar of the nominal integration time; the EF 0691’s integration time is not published, so $1/f_s$ is assumed.
4. The probe correction holds the gain at $g_{\max}$ where $|H|$ is small (Section 6.12); lines there are under-corrected.
5. Spectrogram channel power is absolute only with `--rbw-hz`; the ENBW factor assumes a Gaussian filter unless `--enbw-factor` is given.
6. The significance (false-alarm probability) of Lomb–Scargle peaks is not computed.
7. Content above the Nyquist frequency aliases; nothing in the data can remove it after the fact.

8. Above the True-RMS range the diodes leave square-law operation and powers no longer add; the envelope spectrum is then distorted (a warning is given).
9. The time page plots every sample, so very long logs make large PDFs, and each extra unit section adds its pages again.
10. E and H derived from a power density (and power density from E or H) assume a far-field plane wave, Eq. (6.7); near a source they are only equivalent values. B is never converted.
11. CSV, NPZ and JSON files stay in SI units whatever the display units.
12. Live mode depends on how the instrument software writes its files (Section 9).
13. The GUI and the verification were tested offscreen on Linux; the GUI follows the QuickSyn controller, which runs on Windows.

14 Troubleshooting

Table 22: Troubleshooting.

Symptom	Likely cause	Remedy
'no value column found'	unusual column names	--value-col NAME (or index)
time axis wrong or 'cannot parse'	unusual date format, month/day order	--time-col, --dayfirst; check with --inspect
numbers 1000x too large	decimal comma read as thousands	--decimal , --sep ;
'measured sample rate differs'	logging interval longer than the probe rate	expected with timer logs; Nyquist is then lower
lines at 2f, f1+f2	--quantity field	use the default --quantity power
SignalShark axes look wrong	unknown .h5 layout	--inspect, then --h5-level/--h5-freq/--h5-time
dBm levels, no W/m^2	no antenna factor	--antenna-factor-db AF
--render: 'Veusz not found'	Veusz not on PATH	set VEUSZ_EXE to veusz.exe; the .vszh5 is written anyway
ImportError: vszh5_writer too old	an older copy imported first (Spyder)	restart the kernel; keep 0.0.13 next to the script
'unknown display unit'	misspelt unit	--list-units shows every key; µ, ^2 and dB(...) spellings are accepted
an extra Veusz unit has no pages	its family is not in the record, or the display units already show it	see the display_unit dataset ('no pages'); choose a unit of the record's family
time plot shows the analysed quantity, not E/H	E/H cannot be derived (e.g. dBm without antenna factor)	give --antenna-factor-db, or choose another time-plot mode
--gpu but 'CPU' in backend	no CuPy or CUDA device	mamba install -c conda-forge cupy; NVIDIA only
live: 'waiting for data'	file has no complete row yet, or other layout	check the file grows; --inspect it once
no window opens; help text printed	no Qt binding installed	mamba install -c conda-forge qt.py qtpy pyside6 pyqtgraph
Spyder: window opens and closes at once	graphics backend 'inline'	Preferences > IPython console > Graphics > Qt, or run in a terminal
values show '(x0.001)' on an axis	(fixed in 0.0.3)	update to 0.0.3
channel level too high	oversampled trace summed without RBW	give --rbw-hz (GUI: RBW in Spectrograms)

A Python modules used

This appendix lists every Python module the analyzer uses, the functions, classes and methods it calls, how it calls them and why that module was chosen. Table 23 is the overview; the following subsections give the details, module by module. Versions are those of the tested environment (2026-10-15).

Table 23: Python modules used by the analyzer.

Module	Tested	Needed for	Why this module
`numpy`	2.5.4	all arrays and array maths	common array type of every library used
`scipy`	1.18.1	FFT, windows, CZT, short-time FFT, peaks, interpolation	tested implementations of every estimator needed; scikit-rf has none of them
`pandas`	3.0.6	reading text and Excel exports	fast chunked CSV reader, tolerant date parsing
`openpyxl`	3.1.5	.xlsx (engine of pandas)	pandas' reader for .xlsx
`h5py`	3.16.0	SignalShark .h5; .vszh5 output	direct HDF5 access incl. references
`vszh5_writer`	0.0.13	Veusz documents, rendering	the author's Veusz writer
`Qt.py / QtPy + PySide6`	2.0.7 / 2.4.3 / 6.12	GUI	one Qt API over any binding; PySide6 binding
`pyqtgraph`	0.14.0	GUI plots	fast repeated plot updates, exporters
`cupy`	optional	--gpu	NumPy interface on NVIDIA GPUs
`pypdf`	6.20.1	PDF bookmarks (--render, manual check)	pure Python PDF editing
`veusz`	4.2.1	rendering (--render)	the plotting program of the documents
standard library	3.12.13	CLI, files, processes, time, JSON, ...	no extra dependency

A.1 NumPy

NumPy [20] holds every array of the program. It was chosen because SciPy, pandas, h5py, CuPy and pyqtgraph all exchange data as NumPy arrays, so no conversion is needed anywhere.

Table 24: NumPy calls.

Call	Where (function)	How and why
`np.asarray, np.ascontiguousarray`	everywhere	convert inputs to float64; contiguous arrays for FFTs and HDF5 writes
`np.argsort(kind='stable'), np.unique`	make_uniform, read_signalshark_csv	sort time stamps keeping equal stamps in order, drop duplicates; pivot long-format CSV
`np.diff, np.median, ndarray.mean`	make_uniform, analyse_spectrumset	sample interval = mean of gap-free intervals; jitter = median deviation
`np.interp`	make_uniform, read_ehp_folder	linear interpolation onto the uniform grid / common frequency axis: local, no overshoot, O(N)
`np.memmap, ndarray.tofile`	SpillBuffer, RingBuffer	long columns live in a raw file mapped into memory, so RAM stays bounded; live log on disk
`np.sinc`	probe_response	normalised sinc sin(pi x)/(pi x), the boxcar response of eq. (sinc)
`np.log10, np.maximum`	all dB conversions	dB values with a floor, so log10(0) never occurs
`np.nanmean, np.nanmax, np.nansum, np.where`	analyse_spectrumset	spectrogram statistics with missing cells (NaN) ignored
`np.stack, np.column_stack, np.vstack`	welch_blocks, readers, outputs	batch of Welch segments; tables for CSV; spectra of an EHP folder

Table 24 (continued)

Call	Where (function)	How and why
`np.linspace, np.arange`	czt_zoom, lomb_scargle, synthetic files	frequency grids and time axes
`np.convolve`	_probe_filter (self-test)	boxcar-average synthetic data like a probe
`np.savetxt, np.savez_compressed`	_write_csv, write_outputs	CSV and compressed .npz outputs
`np.random.default_rng`	self-test	reproducible synthetic noise and jitter

A.2 SciPy

SciPy [10] supplies the spectral estimators. It provides, in one tested library, everything the problem needs: real FFTs with multithreading, standard windows, the chirp-Z zoom, the short-time FFT and peak finding. scikit-rf [15] was considered and not used: it is built for network parameters (S-parameters, Touchstone files, calibration), not for spectral analysis of time series.

Table 25: SciPy calls.

Call	Where (function)	How and why
`scipy.fft.rfft(x, n, axis, workers)`	amplitude_spectrum, welch_blocks, analyse_spectrumset	FFT of real data (half the work of a complex FFT); workers = threads (-1 all, 1 per process in batch mode)
`scipy.fft.rfftfreq(n, d)`	same	frequency of every rfft bin
`scipy.signal.get_window(name, N)`	every estimator	any SciPy window by name (--window); periodic form (fftbins=True), correct for spectral analysis
`scipy.signal.zoom_fft(x, [f1, f2], m, fs, endpoint=True)`	czt_zoom	chirp-Z zoom (Bluestein), dense spectrum samples in a band
`scipy.signal.ShortTimeFFT(win, hop, fs, fft_mode='onesided2X', scale_to='psd')`	envelope_spectrogram	spectrogram(detr='constant', p0, p1), lower_border_end, upper_border_begin, t(), f: one-sided PSD slices without the slices hanging over the ends (SciPy >= 1.12)
`scipy.signal.find_peaks(x, height, prominence)`	find_spectral_peaks, analyse_spectrumset	local maxima above a level and prominence: envelope lines; carriers for channels
`scipy.signal.welch, periodogram, spectrogram, lombscargle`	verify_transforms, self_test	independent reference implementations the program's estimators are checked against
`scipy.interpolate.interp1d(x, y, axis=0)`	analyse_spectrumset	all frequency bins of a spectrogram onto a uniform time grid in one call (a legacy SciPy class, still supported)

A.3 pandas and openpyxl

pandas [21] reads the text and Excel exports. Its CSV reader is fast (C engine), reads in chunks, and its date parser copes with the many date and time formats instrument software writes [33]. openpyxl [34] is not imported by the program; pandas uses it as the engine for `.xlsx` files.

Table 26: pandas calls.

Call	Where (function)	How and why
`pd.read_csv(sep, decimal, encoding, skiprows, header=None, usecols, dtype=str, chunksize, engine, on_bad_lines='skip')`	read_text_timeseries, read_text_spectrum, read_signalshark_csv	reads only the needed columns as text, in chunks (bounded memory); bad lines skipped; numbers converted afterwards so decimal commas and thousands dots are handled
`Series.str.strip/.replace/.match`	_num, _to_seconds	clean quotes and spaces; decimal comma
`pd.to_numeric(errors='coerce')`	_num, _to_seconds	text to numbers, NaN where not a number

Table 26 (continued)

Call	Where (function)	How and why
`pd.to_datetime(format='mixed', dayfirst, errors='coerce')`	_to_seconds, LiveTail.poll	date/time strings of any common format; .dt.tz_convert(None), .dt.total_seconds() give seconds
`pd.read_excel(sheet_name=0, header=None, dtype=str)`	read_xlsx_timeseries	first sheet as text (openpyxl engine), then the CSV reader path
`DataFrame.to_csv(sep=';')`	read_xlsx_timeseries	temporary CSV of the sheet

A.4 h5py

h5py [22] reads SignalShark `.h5` files and, through `vszh5_writer.py`, writes the Veusz `.vszh5` documents, which are HDF5 files. It gives direct access to groups, datasets, attributes and object references, which the Veusz format needs.

Table 27: h5py calls.

Call	Where (function)	How and why
`h5py.File(path, 'r')`	read_signalshark_h5, describe_h5, self_test	open HDF5 read-only
`File.visititems(callback)`	read_signalshark_h5, describe_h5	walk every group and dataset to find level and axis datasets / print the tree
`Dataset.ndim/.shape/.dtype, .attrs, .parent`	read_signalshark_h5	choose datasets by shape and type; gather attributes up to the root (units, start/stop)
`h5py.File(path, 'w'), create_group, create_dataset(compression='gzip', compression_opts=4, shuffle=True)`	vszh5_writer.VeuszWriter	the .vszh5 layout, gzip for arrays of 4096+ values
`Dataset.ref (object reference)`	vszh5_writer.VeuszWriter.dataset_2d	links an image to its axis arrays (vsz_xrange etc.), as Veusz itself does
`File['Veusz/Data/<name>/data'][()], File['Veusz/Document/document']`	vszh5_writer.read_text_dataset, page_names, self_test	reading a written document back: text datasets, page order, stored values (0.0.4)

A.5 vszh5_writer (the author's module, 0.0.13)

`vszh5_writer.py` writes Veusz's native HDF5 document format without Veusz, and renders it to PDF and PNG when Veusz is installed. It came from the author's `isp_speed_monitor.py` and the QuickSyn package (0.0.10); for this program it was extended to 0.0.11 (two-dimensional datasets, line and image page helpers), 0.0.12 (overview grid pages and text pages) and 0.0.13 (reading a written document back: dataset names, text datasets, page order), without changing any existing call; the 2-D layout follows the Veusz source [29]. The analyzer uses it for every Veusz document, so the pages look the same as in the author's other programs.

Table 28: vszh5_writer calls.

Call	Where (function)	How and why
`VeuszWriter(path, compress=True, writer_tag)`	write_vszh5_series, write_vszh5_spectrogram	context manager; writes <path>.part, publishes with finish(), deletes on error (abort)
`dataset_1d / dataset_date / dataset_text`	_write_series_pages	typed Veusz datasets: numbers, dates (seconds since 2009-01-01), the JSON summary
`dataset_2d(name, data, xcent=, ycent=)`	_write_series_pages, write_vszh5_spectrogram	images; regular centres stored as xrange/yrange like Veusz (new in 0.0.11)
`VszDoc().text()`	write_vszh5_*	the Veusz command script stored in the document
`Trace(x, y, label, style, marker, line)`	_write_series_pages	one line or marker series of a page (new in 0.0.11)

Table 28 (continued)

Call	Where (function)	How and why
`xy_page(doc, page, traces, title, xlabel, ylabel, xlog, ylog, xmin, xmode, x_autorange)`	_write_series_pages	a complete line-plot page with title and legend (new in 0.0.11)
`image_page(doc, page, dataset, title, ..., zmin, zmax)`	_write_series_pages, write_vszh5_spectrogram	a complete image page with colour bar (new in 0.0.11)
`render_pages(path, png_pages='all', bookmarks=callable, pdf_info)`	render_vszh5	PDF and PNGs through Veusz; bookmarks from bookmark_outline()
`Panel, grid_page(doc, page, panels, title)`	_write_series_pages	overview page: time series, amplitude, Welch and zoom on one page (new in 0.0.12)
`text_page(doc, page, title, lines)`	write_vszh5_series, write_vszh5_spectrogram	summary page of units, timing, peaks, families and notes (new in 0.0.12)
`TRACE_COLOURS, find_veusz_exe()`	PlotPanel, MainWindow.open_in_veusz	the same colours in the GUI as on the Veusz pages; locating Veusz
`dataset_text('display_unit' / 'unit_sections', ...)`	_units_dataset	the units of every section stored in the document (new in 0.0.4)
`read_text_dataset(path, 'display_unit')`	render_vszh5	section titles read back for the bookmarks of a document written earlier (new in 0.0.13)
`dataset_names(path), page_names(path)`	self_test, build_manual_pdf.bookmark_example	check a multi-unit document; the bookmark tree shown in this manual (new in 0.0.13)

A.6 Qt.py, QtPy, PySide6 and pyqtgraph

The GUI is written against a Qt abstraction layer, a thin module that exposes one Qt API over PySide6, PyQt6 and the older bindings, so the same source runs on any of them: Qt.py [24] first (as in the QuickSyn controller), QtPy [40] as the alternative (`EMF_FFT_QT_LAYER=qtpy`), PySide6 [25] directly as the last resort; the binding asked for is PySide6 (`QT_PREFERRED_BINDING`, `QT_API`). Without any binding the module still imports (the GUI classes are defined on a placeholder) and only the GUI reports what is missing. pyqtgraph [26] draws every plot: it is built for fast, repeated updates of large arrays, which live display and long logs need, and runs on the same binding.

Table 29: Qt.py, PySide6 and pyqtgraph calls.

Call	Where (function)	How and why
`from Qt import ... / from qtpy import ... / from PySide6 import ...`	_import_qt	abstraction layer in order Qt.py, QtPy, PySide6; binding PySide6 (QT_PREFERRED_BINDING, QT_API)
`QApplication(), .setStyle('Fusion'), .setPalette(), .exec()`	launch_gui, ThemeManager.apply	event loop; Fusion style so a palette themes every widget; Light/Dark
`QMainWindow.menuBar(). addMenu, .statusBar(), setCentralWidget`	MainWindow	File/View/Analysis menus and the status line
`QActionGroup (QtGui in Qt 6, QtWidgets in Qt 5)`	MainWindow._build_menus	exclusive Light/Dark choice; looked up in either module
`QtCore.QTimer.timeout, QTimer.singleShot`	LivePanel, launch_gui	periodic update; --live-seconds
`QtCore.QThread, QtCore.Signal (messageLogged, resultDelivered, ...)`	TaskThread	analysis, inspect, verification and self-test off the GUI thread; results handed to the window through queued signals
`QtCore.QSettings(org, app)`	MainWindow	options, theme, geometry, recent inputs between sessions
`QMainWindow.addDockWidget, QDockWidget.toggleViewAction`	MainWindow	settings dock (left) and log dock (bottom), shown or hidden from View
`QMainWindow.addToolBar, QToolBar.addWidget`	_build_toolbar	main toolbar with the unit selector

Table 29 (continued)

Call	Where (function)	How and why
`QFileDialog.getOpenFileNames/getExistingDirectory/getSaveFileName`	SettingsPanel, MainWindow	choosing inputs, folders, export and settings files
`QMessageBox (About, warning, question)`	build_about_box, MainWindow	About dialog with rich text and Show Details; warnings; quit confirmation
`dragEnterEvent/dropEvent, QMimeData.urls`	MainWindow	drag and drop of files and folders
`QDesktopServices.openUrl`	MainWindow	open the manual, output folder and rendered PDF
`QApplication.clipboard().setText`	MainWindow	copy summary, peaks and log
`QtSvg.QSvgGenerator + QWidget.render`	export_widget_svg	vector SVG of a plot when pyqtgraph's SVG exporter fails
`QtGui.QPalette, QColor, ColorRole/ColorGroup`	ThemeManager	dark and light palettes (System restores the default)
`QListWidgetItem.setFlags(ItemIsUserCheckable), setCheckState`	SettingsPanel	tick list of the extra Veusz units (0.0.4)
`QComboBox.findData/currentData per unit family`	SettingsPanel.unit_combos, MainWindow	one unit selector per family, mirrored in View > Units and on the toolbar (0.0.4)

Table 30: pyqtgraph calls.

Call	Where (function)	How and why
`pg.PlotWidget, PlotItem.plot(..., connect='finite')`	PlotPanel	every line plot; NaN breaks the line (gaps, non-positive values on log axes)
`PlotItem.setDownsampling(auto=True, mode='peak'), setClipToView`	PlotPanel	long logs stay fast: only visible data, peak-preserving decimation
`AxisItem.enableAutoSIPrefix(False)`	PlotPanel, ImagePanel	values read in the unit named on the axis (no automatic x0.001)
`pg.DateAxisItem(utcOffset=0), PlotItem.setAxisItems`	PlotPanel.use_date_axis	instrument wall-clock time on the x axis when the start time is known
`PlotItem.setLogMode, addLegend, LegendItem.setLabelTextColor`	PlotPanel	log amplitude axes, legends, theme colours
`pg.ImageItem(axisOrder='row-major'), setRect, pg.ColorBarItem, pg.colormap.get('viridis')`	ImagePanel	spectrograms and modulation maps with a colour bar
`scene().sigMouseMoved, ViewBox.mapSceneToView`	PlotPanel, ImagePanel	cursor readout in the status bar
`exporters.ImageExporter, SVGExporter, CSVExporter`	PlotPanel.export, ImagePanel.export	File > Export current plot
`pg.mkPen(colour, width, style)`	PlotPanel.add_curve	solid, dashed and dotted traces

A.7 CuPy (optional)

CuPy [27] offers the NumPy interface on NVIDIA GPUs, so the same code runs on either device through the program's `Backend` class (its `xp` attribute is `numpy` or `cupy`). It is imported only when `--gpu` is given.

Table 31: CuPy calls (only with –gpu).

Call	Where (function)	How and why
`cupy.cuda.runtime.getDeviceCount()`	Backend	is a CUDA device present?

Table 31 (continued)

Call	Where (function)	How and why
`cupy.asarray, cupy.fft.rfft`	Backend.asarray, Backend.rfft	arrays and FFTs on the GPU
`ndarray.get()`	Backend.cpu	copy results back to host memory
`get_default_memory_pool(). free_all_blocks(), pinned pool`	flush_memory	release GPU memory

A.8 pypdf and Veusz (rendering only)

Neither is imported by the analyzer itself; `vszh5_writer.py` uses them when `--render` is given, and the manual builder uses pypdf [28] to check the bookmarks of this PDF.

Table 32: pypdf and Veusz (through vszh5_writer).

Call	Where (function)	How and why
`veusz --listen --quiet ( script on stdin)`	vszh5_writer.render_ pages	loader.loadHDF5Doc then Export(): veusz --export cannot open .vszh5 files
`pypdf.PdfWriter(clone_from), add_outline_item, add_ metadata, page_mode`	add_pdf_bookmarks	bookmark tree and document information written into the rendered PDF
`pypdf.PdfReader(...).outline`	build_manual_pdf. verify_bookmarks	reads this manual's bookmarks back to check them

A.9 Python standard library

Table 33: Python standard library modules used.

Module	Calls	Purpose
`argparse`	ArgumentParser, add_argument	command line (also read by the manual builder)
`atexit`	register	delete spill files when the program exits
`concurrent.futures`	ProcessPoolExecutor, as_completed	--jobs: files in parallel processes
`csv`	reader	split lines with quoted fields
`dataclasses`	dataclass (frozen), field, asdict	Options, TimeSeries, results, DisplayUnit, UnitView
`types`	SimpleNamespace	a live update as a record for the unit helpers; verification
`datetime`	datetime, timedelta, timezone	time origins, EHP file times, Veusz dates
`gc`	collect	flush_memory()
`json`	dumps	summary file and summary dataset
`math`	floor, ceil, log2, sqrt, isfinite	segment lengths, scalars
`os`	fdopen, unlink, replace, environ	spill files, atomic writes, Qt binding choice
`pathlib`	Path	all file paths (Windows and Linux)
`pickle`	dump (protocol 5), load	--pickle offload of results
`re`	search, match, fullmatch, compile	units, headers, file names, bookmarks
`sys`	path, modules, argv, exit	module path for vszh5_writer, CuPy check, CLI
`tempfile`	mkstemp, mkdtemp	spill files, xlsx conversion, self-test folder
`time`	perf_counter, time, sleep	timing reports, live polling
`typing`	Any, Sequence	type hints
`warnings`	warn(RuntimeWarning)	render problems reported without stopping
`subprocess, threading`	Popen, Timer	vszh5_writer: drive Veusz with a timeout
`textwrap, html, platform`	wrap, escape, python_version	summary page, About dialog
`importlib`	import_module	QtSvg through the same Qt layer
`contextlib, io`	redirect_stdout, StringIO	self-test report into the GUI log

B Revision history

Table 34: Revision history of the program, the writer module and this manual.

Version	Date	Changes
`0.0.1`	2026-10-15	First draft of the analyzer: readers, FFT, Welch, CZT, Lomb-Scargle, spectrogram, probe model and correction, harmonic families, per-bin modulation, .vszh5 output, batch, GPU option, live mode, self-test.
`0.0.2`	2026-10-15	Readability pass (descriptive names, detailed comments; numerical results identical). Veusz output through vszh5_writer 0.0.11; date axis on time pages; zoom page with FFT bins; hierarchical PDF bookmarks; summary keys rf_bins/rf_range_hz renamed frequency_bins/frequency_range_hz; fuller dark palette. This manual and its build tool.
`writer 0.0.11`	2026-10-15	vszh5_writer: dataset_2d, Trace, add_title, xy_page, image_page (manual colour bar, exact ranges), date axes; existing calls unchanged.
`0.0.3`	2026-10-15	GUI (default entry; Qt.py/QtPy over PySide6; QuickSyn-style layout, themes, menus, toolbar, About with author/version/GPL, exports, live tab, verification tab). Display units for power density. verify_transforms (44 checks). Fixes found by it: Nyquist row of per-bin modulation spectra, unit headers mW/m2, uW/m2, dB(uV/m), dBm/m2, exact dBm-dBuV constant, continuous probe-correction gain. RBW channel power. Summary and overview pages. Manual: GUI chapter, detailed methods, verification table.
`writer 0.0.12`	2026-10-15	vszh5_writer: Panel and grid_page (several graphs on one page), text_page (pages of text); existing calls unchanged.
`0.0.4`	2026-10-15	Unit control: display units for every family (power density, E, H, B, port power and voltage; linear and dB; squared fields), one per family; time plots as power density, E, H, logged field or analysed quantity; RF levels as power density, E or H; Veusz in several units (sections or files) in GUI, batch and live snapshots; Veusz > Write .vszh5 again; --list-units; 10 new verification checks (54); Clear buffer and Free memory. Manual: units section with derivations, multi-unit output, new figures.
`writer 0.0.13`	2026-10-15	vszh5_writer: dataset_names, read_text_dataset and page_names read a written document back without Veusz; existing calls unchanged.

References

[1] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-4091,” NSTS 0523-E0399A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/48a83a03-af45-41d0-820c-ecdb46ba1437/Narda_EFD-4091-datasheet.pdf

[2] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-0691,” NSTS 0624-E0387B, 2024. [Online]. Available: https://www.atecorp.com/getmedia/84e296e2-1bd2-4861-8a1f-e905f39d4910/Datasheet_EFD-0691.pdf

[3] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-9091,” NSTS 0523-E0405A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/c56e0776-d855-4d49-9081-ff316ecaf68a/Datasheet_EFD-9091.pdf

[4] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-1891,” NSTS 0723-E0398A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/d5ed148e-2dfc-4f3c-afa4-2b306fb7a781/Narda_EFD-1891-datasheet.pdf

[5] Narda Safety Test Solutions GmbH, “Datasheet Narda FieldMan,” NSTS 0925-E0370D, 2025. [Online]. Available: https://www.atecorp.com/getmedia/5511a8ac-b386-434f-bac9-fd22d18e2f48/Narda-Fieldman-Datasheet-2025.pdf

[6] Narda Safety Test Solutions, “NBM-550 Remote Control,” V1.0. [Online]. Available: https://wiki.radimation.com/wiki/images/7/7b/NBM-550_Remote_Control_V1_0.pdf

[7] Narda Safety Test Solutions S.r.l., “EHP-50F User’s Manual,” EHP50FEN-10107-1.73, 2021. [Online]. Available: https://atecorp.com/getmedia/d53b58ae-844a-4a2d-b84e-6a9f98be805a/narda-ehp-50f-operating-manual.pdf

[8] Narda Safety Test Solutions, “SignalShark options and scripts.” [Online]. Available: https://www.narda-sts.com/en/products/receivers-/-direction-finders-/-spectrum-analyzers/signalshark-optionen/

[9] Narda Safety Test Solutions, “PC software Narda-TSX.” [Online]. Available: https://www.narda-sts.com/en/products/emc-products-and-streaming-solutions/pc-software-narda-tsx/

[10] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[11] P. D. Welch, “The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms,” *IEEE Trans. Audio Electroacoust.*, vol. AU-15, no. 2, pp. 70–73, Jun. 1967, doi: 10.1109/TAU.1967.1161901.

[12] L. R. Rabiner, R. W. Schafer, and C. M. Rader, “The chirp z-transform algorithm,” *IEEE Trans. Audio Electroacoust.*, vol. AU-17, no. 2, pp. 86–92, Jun. 1969, doi: 10.1109/TAU.1969.1162034.

[13] J. D. Scargle, “Studies in astronomical time series analysis. II. Statistical aspects of spectral analysis of unevenly spaced data,” *Astrophys. J.*, vol. 263, pp. 835–853, 1982, doi: 10.1086/160554.

[14] J. T. VanderPlas, “Understanding the Lomb–Scargle periodogram,” *Astrophys. J. Suppl. Ser.*, vol. 236, no. 1, 2018, doi: 10.3847/1538-4365/aab766.

[15] scikit-rf developers, “scikit-rf documentation.” [Online]. Available: https://scikit-rf.readthedocs.io/

[16] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978, doi: 10.1109/PROC.1978.10837.

[17] *IEEE Standard for Calibration of Electromagnetic Field Sensors and Probes (Excluding Antennas) from 9 kHz to 40 GHz*, IEEE Std 1309-2013. [Online]. Available: https://standards.ieee.org/ieee/1309/5031

[18] *Determination of RF field strength, power density and SAR in the vicinity of base stations for the purpose of evaluating human exposure*, IEC 62232:2025. [Online]. Available: https://webstore.iec.ch/en/publication/89073

[19] L. I. Bluestein, “A linear filtering approach to the computation of discrete Fourier transform,” *IEEE Trans. Audio Electroacoust.*, vol. 18, no. 4, pp. 451–455, Dec. 1970, doi: 10.1109/TAU.1970.1162132.

[20] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[21] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[22] h5py developers, “HDF5 for Python (h5py) documentation.” [Online]. Available: https://docs.h5py.org/

[23] J. Sanders *et al.*, “Veusz — a scientific plotting package.” [Online]. Available: https://veusz.github.io/

[24] M. Ottosson *et al.*, “Qt.py: Minimal Python 2 & 3 shim around all Qt bindings.” [Online]. Available: https://github.com/mottosso/Qt.py

[25] The Qt Company, “Qt for Python (PySide6) documentation.” [Online]. Available: https://doc.qt.io/qtforpython-6/

[26] pyqtgraph developers, “PyQtGraph: Scientific graphics and GUI library for Python.” [Online]. Available: https://www.pyqtgraph.org/

[27] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems (LearningSys), 31st Conf. Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: http://learningsys.org/nips17/assets/papers/paper_16.pdf

[28] pypdf developers, “pypdf documentation.” [Online]. Available: https://pypdf.readthedocs.io/

[29] Veusz developers, “Veusz 4.2.1 source (document/doc.py, document/loader.py, datasets/twod.py),” 2024. [Online]. Available: https://pypi.org/project/veusz/4.2.1/

[30] SciPy developers, “scipy.signal.ShortTimeFFT.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.ShortTimeFFT.html

[31] SciPy developers, “scipy.signal.zoom_fft.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.zoom_fft.html

[32] P. J. Mohr, D. B. Newell, B. N. Taylor, and E. Tiesinga, “CODATA recommended values of the fundamental physical constants: 2022,” arXiv:2409.03787, 2024. [Online]. Available: https://arxiv.org/abs/2409.03787

[33] pandas developers, “pandas documentation.” [Online]. Available: https://pandas.pydata.org/docs/

[34] openpyxl developers, “openpyxl documentation.” [Online]. Available: https://openpyxl.readthedocs.io/

[35] Spyder project, “Spyder — the scientific Python development environment.” [Online]. Available: https://www.spyder-ide.org/

[36] conda-forge, “Miniforge (conda and mamba).” [Online]. Available: https://github.com/conda-forge/miniforge

[37] H. Oberdiek and the LaTeX Project, “The bookmark package.” [Online]. Available: https://ctan.org/pkg/bookmark

[38] N. R. Lomb, “Least-squares frequency analysis of unequally spaced data,” *Astrophys. Space Sci.*, vol. 39, pp. 447–462, 1976, doi: 10.1007/BF00648343.

[39] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[40] QtPy developers (Spyder project), “QtPy: Provides an abstraction layer on top of the various Qt bindings.” [Online]. Available: https://github.com/spyder-ide/qtpy

[41] *Letter symbols to be used in electrical technology — Part 3: Logarithmic and related quantities, and their units*, IEC 60027-3:2002, ed. 3.0, Jul. 2002. [Online]. Available: https://webstore.iec.ch/en/publication/94

[42] *Quantities and units — Part 1: General*, ISO 80000-1:2022. [Online]. Available: https://www.iso.org/standard/76921.html